MATH70099 – Big Data: Statistical Scalability with PySpark

Coursework 2

CID: 06060027

Contents

Reproducibility. All results are fully reproducible with a single `SEED = 6060027` (CID). Environment: Python 3.10, PySpark 3.3.1, NumPy 1.24, SciPy 1.10, Matplotlib 3.7. Q1 is intended for Spark/HDFS on Bazooka; Q2–Q3 use the Athena local file system; Q4 uses PySpark on Athena via SSH. The active source files are `src/q1_window_functions.py`, `src/q2_bayes_precision.py`, `src/q3_markov_vi_ewma.py`, and `src/q4_pyspark_public_dataset.py`. The report figures are compiled from the synced outputs in `../outputs_athena/`.

1 Question 1: Window Functions [20 marks]

1.1 Part (a): Data Loading and Time Columns

We define a global elapsed-time variable and a jittered version to ensure a well-defined temporal ordering of events.

The original timestamps are converted from seconds to minutes and re-centred by subtracting the global minimum, so that the time domain starts at zero. This yields a variable consistent with the standard Poisson-process formulation $t \in [0, T]$.

To avoid ties in the observed times, we introduce a perturbed variable of the form

$$\texttt{time} = \texttt{time\_elapsed} + U, \quad U \sim \text{Unif}[0, 1),$$

generated via `F.rand(seed=6060027)`. This guarantees that observations are almost surely distinct, ensuring that lag-based inter-arrival computations are well-defined, while the magnitude of the perturbation is negligible relative to typical time scales and does not affect the modelling assumptions.

```
bikes = spark.read.csv("/shared_data/santander_trips.csv",
                       header=True, inferSchema=True)

min_time = bikes.agg(F.min("start_time")).collect()[0][0]
bikes = (bikes
    .withColumn("time_elapsed",
                (F.col("start_time") - F.lit(min_time)) / 60.0)
    .withColumn("time",
                F.col("time_elapsed") + F.rand(seed=6060027)))
```

Listing 1: Part (a): data loading and time columns

1.2 Part (b): Estimating Arrival Rates

For each source station i, departures are modelled as a homogeneous Poisson process with rate λ_i. The question sheet gives the MLE

$$\hat{\lambda}_i = \frac{N_i(T)}{T},$$

where $N_i(T)$ is the number of trips started from station i over the common observation window $[0, T]$.

In our setting, T is the common horizon given by the largest value of `time_elapsed`, namely

$$T = \max(\texttt{time\_elapsed}) = 161\,274.0$$

minutes, while $N_i(T)$ is simply the number of trips whose source station is i. Hence the estimator is obtained by counting, for each `start_id`, the number of observed departures and dividing by the common horizon T.

Using a Window partitioned by `start_id`, the station-specific counts are attached directly to each row of the DataFrame, after which `lambda_hat` is computed as $N_i(T)/T$.

```
T = bikes.agg(F.max("time_elapsed")).collect()[0][0]

station_window = Window.partitionBy("start_id")
bikes = (bikes
    .withColumn("N_i", F.count("*").over(station_window).cast(DoubleType()))
    .withColumn("lambda_hat", F.col("N_i") / T))

# Report 5 largest and smallest lambda_hat
station_df = bikes.select("start_id", "lambda_hat").dropDuplicates(["start_id"])
station_df.orderBy(F.desc("lambda_hat")).show(5)
station_df.orderBy(F.asc("lambda_hat")).show(5)
```

Listing 2: Part (b): $\hat{\lambda}_i$ via a Window function

The estimated rates vary substantially across stations, indicating strong heterogeneity in journey intensity across the network. The largest values correspond to major hubs with frequent departures, whereas the smallest values are associated with comparatively quiet stations.

	`start_id`	$\hat{\lambda}_i$ (trips/min)
Largest	191	0.2161
	785	0.1653
	213	0.1364
	307	0.1295
	248	0.1289
Smallest	494	0.00239
	608	0.00303
	842	0.00430
	767	0.00673
	472	0.00679

Table 1: Five stations with the largest and smallest estimated Poisson rates.

1.3 Part (c): Inter-arrival p-values

Under the Poisson-process assumption, the inter-arrival times at station i,

$$\Delta t_{i,j} = t_{i,j} - t_{i,j-1},$$

follow an exponential distribution with rate λ_i. Replacing λ_i with the estimate $\hat{\lambda}_i$, we compute the corresponding right-tail probability

$$p_{i,j} = \exp\big(-\hat{\lambda}_i \Delta t_{i,j}\big),$$

with $t_{i,0} = 0$. Values approximately consistent with a uniform distribution on $[0, 1]$ indicate agreement between the observed inter-arrival times and the fitted exponential model.

In practice, the inter-arrival times are obtained by ordering observations within each station and applying a lag operator. Using a Window partitioned by `start_id` and ordered by `time`, the previous event time is retrieved via `lag`, and the resulting differences are used to compute the column `pval`.

```
time_window = Window.partitionBy("start_id").orderBy("time")
bikes = (bikes
    .withColumn("prev_time",
                F.lag("time", 1, 0).over(time_window))
    .withColumn("inter_arrival",
                F.col("time") - F.col("prev_time"))
    .withColumn("pval",
                F.exp(-F.col("lambda_hat") * F.col("inter_arrival"))))

# Report 5 stations with largest/smallest mean p-value
avg_pval = bikes.groupBy("start_id").agg(F.mean("pval").alias("avg_pval"))
avg_pval.orderBy(F.desc("avg_pval")).show(5)
avg_pval.orderBy(F.asc("avg_pval")).show(5)
```

Listing 3: Part (c): inter-arrival p-values via Window + lag

The mean p-value for each station provides a simple summary of how closely the observed inter-arrival times agree with the fitted exponential model. Values closer to 0.5 are more compatible with approximate uniformity, whereas larger deviations suggest a poorer fit.

	start_id	Mean p-value
Largest	842, 794, 841, 840, 354	0.875, 0.822, 0.810, 0.807, 0.772
Smallest	686, 208, 165, 328, 565	0.582, 0.586, 0.587, 0.587, 0.589

Table 2: Stations with the largest and smallest average p-values.

1.4 Part (d): Kolmogorov–Smirnov Statistics

To assess the goodness of fit of the Poisson model, we compare the distribution of the station-specific p-values with the uniform distribution on $[0, 1]$. For station i, the Kolmogorov–Smirnov statistic is

$$D_i = \sup_{x \in [0,1]} \left| F(x) - \hat{F}_i(x) \right|,$$

where $F(x) = x$ is the CDF of the uniform distribution and $\hat{F}_i(x)$ is the empirical cumulative distribution function of the p-values at station i.

Following the instructions in the question, $\hat{F}_i(x)$ is estimated pointwise by applying `percent_rank()` over a Window partitioned by `start_id` and ordered by `pval`. This produces the column `empirical_quantiles`, which provides a pointwise estimate of the empirical CDF at the observed p-values. The absolute difference between `pval` and `empirical_quantiles` is then computed row by row, and the KS score D_i is obtained by taking the maximum of these differences within each station.

```
pval_rank_window = Window.partitionBy("start_id").orderBy("pval")

bikes = (bikes
    .withColumn("empirical_quantiles",
                F.percent_rank().over(pval_rank_window))
    .withColumn("abs_diff",
                F.abs(F.col("pval") - F.col("empirical_quantiles"))))

ks_df = bikes.groupBy("start_id").agg(F.max("abs_diff").alias("ks_stat"))
```

Listing 4: Part (d): KS statistic via `percent_rank`

The resulting KS statistics summarise, for each station, the largest discrepancy between the fitted p-value distribution and the uniform benchmark. Smaller values of D_i indicate closer agreement with the Poisson model, whereas larger values indicate greater departure from the homogeneous Poisson-process assumption.

To visualise the variability of these discrepancies across the network, the station-level KS scores are converted to Pandas and displayed with a boxplot. As an additional sanity check, Appendix A compares empirical inter-arrival CDFs with the fitted exponential CDF for an extreme high-rate station and an extreme low-rate station.

```
ks_values = [row["ks_stat"] for row in ks_df.collect()]
ks_arr = np.array(ks_values, dtype=float)

fig, ax = plt.subplots(figsize=(6, 5))
ax.boxplot(ks_arr, vert=True)
ax.set_ylabel("KS statistic")
ax.set_title("Kolmogorov-Smirnov scores across stations")
plt.show()
```

Listing 5: Part (d): boxplot of KS statistics

Discussion. The statistic

$$D_i = \sup_{x \in [0,1]} \left| \hat{F}_i(x) - x \right|$$

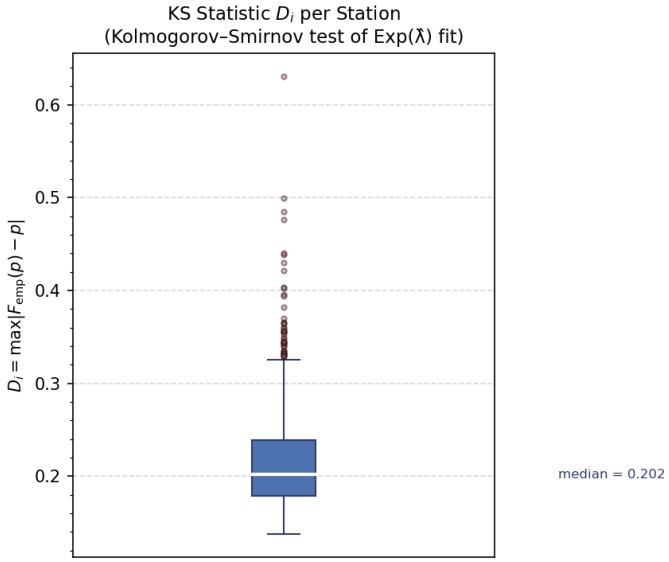


Figure 1: Boxplot of Kolmogorov–Smirnov statistics D_i across source stations.

measures the largest pointwise discrepancy between the empirical CDF of the transformed inter-arrival p-values and the uniform CDF. If the fitted exponential model were adequate, then by the probability integral transform the p-values would be approximately $\text{Unif}(0, 1)$, so D_i should remain small; in the classical KS scaling one expects $D_i = O_p(n_i^{-1/2})$ for station size n_i. The boxplot therefore suggests that the homogeneous Poisson assumption is reasonable for many stations but not exact across the full network, with some stations showing visibly larger deviations. This is consistent with the lecture-note discussion of goodness-of-fit diagnostics based on transformed inter-arrival times for point-process models. As a final sanity check, Appendix A compares the empirical inter-arrival CDF with the fitted exponential CDF for two extreme stations.

2 Question 2: Bayesian Inference on a Precision Matrix [25 marks]

2.1 Part (a): Consensus Monte Carlo

We consider the Gaussian–Wishart model with prior $\mathbf{\Lambda} \sim \mathcal{W}_d(\nu, \mathbf{V})$, where $\nu = d = 6$ and $\mathbf{V} = \mathbf{I}_d$. By conjugacy, the full posterior is

$$\mathbf{\Lambda} \mid \mathbf{X} \sim \mathcal{W}_d\big(\nu + n,\ (\mathbf{I} + S)^{-1}\big), \qquad S = \sum_{i=1}^{n} \mathbf{x}_i \mathbf{x}_i^\top.$$

Sub-posteriors via fractional priors. The data are split across $B = 5$ machines with batch sizes n_b. To ensure that the product of sub-posteriors recovers the full posterior, we assign each batch a fractional prior $p_b(\mathbf{\Lambda}) \propto p(\mathbf{\Lambda})^{n_b/n}$.

For batch b, with scatter matrix $S_b = \sum_{i \in b} \mathbf{x}_i \mathbf{x}_i^\top$, the likelihood is

$$p(\mathbf{X}_b \mid \mathbf{\Lambda}) \propto |\mathbf{\Lambda}|^{n_b/2} \exp\big(-\tfrac{1}{2}\mathrm{tr}(S_b \mathbf{\Lambda})\big).$$

Combining with the fractional prior and matching to the Wishart kernel yields

$$\mathbf{\Lambda} \mid \mathbf{X}_b \sim \mathcal{W}_d(\delta_b, \Psi_b^{-1}),$$

with

$$\delta_b = (\nu - d - 1)\tfrac{n_b}{n} + (d + 1) + n_b, \qquad \Psi_b = \tfrac{n_b}{n}\mathbf{I}_d + S_b.$$

With $\nu = d = 6$, this simplifies to

$$\delta_b = 7 - \tfrac{n_b}{n} + n_b.$$

Sampling. We draw $S = 1000$ samples independently from each sub-posterior using `scipy.stats.wishart.rvs(df=`δ_b`, scale=`Ψ_b^{-1}`)`, and from the full posterior using `wishart.rvs(df=506, scale=`$(I+S)^{-1}$`)`.

CMC combination schemes. Let $\mathbf{\Lambda}^{(b,s)}$ denote the s-th draw from batch b.

(i) Sample-size weighting

$$\tilde{\mathbf{\Lambda}}^{(s)} = \sum_{b=1}^{B} \frac{n_b}{n}\, \mathbf{\Lambda}^{(b,s)}.$$

(ii) Inverse-variance weighting (element-wise)

$$\hat{\Lambda}_{jk}^{(s)} = \frac{\sum_b \Lambda_{jk}^{(b,s)} / \hat{\Omega}_{b,jk}}{\sum_b 1/\hat{\Omega}_{b,jk}}, \qquad \hat{\Omega}_{b,jk} = \widehat{\mathrm{Var}}_s\big[\Lambda_{jk}^{(b,s)}\big].$$

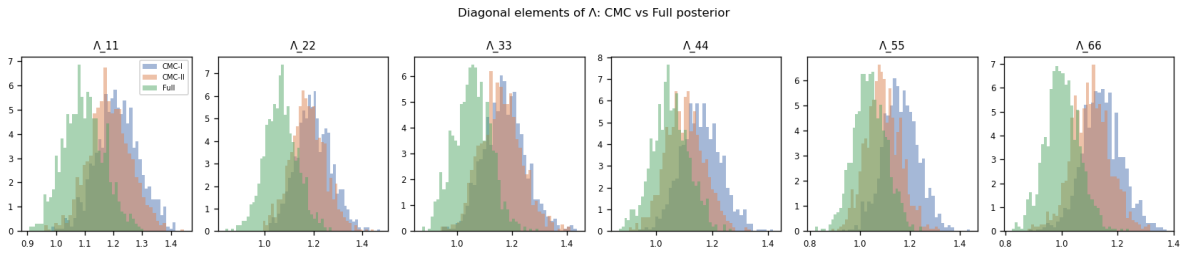


Figure 2: Selected marginal comparisons for the diagonal entries $\Lambda_{11}, \ldots, \Lambda_{66}$. In every panel the full posterior is shifted left of both CMC approximations; scheme (ii) lies between the full posterior and scheme (i), and is therefore closer in location, although both CMC approximations retain slightly heavier right tails.

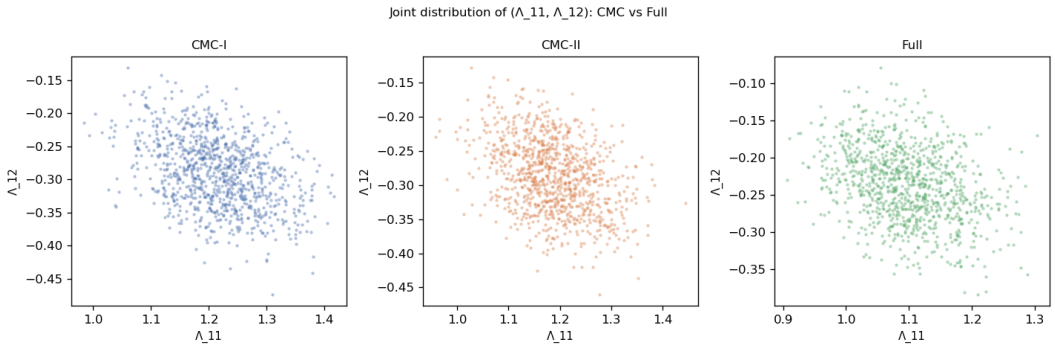


Figure 3: Joint comparison for $(\Lambda_{11}, \Lambda_{12})$. The three methods show a similar negative dependence pattern, but the CMC clouds are displaced relative to the full posterior, with scheme (ii) visually closer than scheme (i).

Results. The comparison with the full posterior is shown in Figures 2 and 3. The diagonal marginals in Figure 2 show the same pattern across $\Lambda_{11}, \ldots, \Lambda_{66}$: the full posterior is the left-most distribution, scheme (i) is typically the most right-shifted, and scheme (ii) lies in between. Hence scheme (ii) gives the closer approximation to the full posterior location for the diagonal entries, although neither approximation matches the full posterior exactly and both keep somewhat heavier right tails.

The joint comparison in Figure 3 leads to the same conclusion. All three clouds show a similar negative association between Λ_{11} and Λ_{12}, so the qualitative dependence structure is preserved. However, scheme (i) is visibly more displaced relative to the full posterior, while scheme (ii) is closer in centre and spread. Hence the visual evidence supports the ordering

$$\text{full posterior} \quad \approx \quad \text{scheme (ii)} \quad \text{better than} \quad \text{scheme (i)},$$

with both CMC constructions remaining approximate.

Discussion. The two weighting schemes are therefore only partially appropriate in this setting. Scheme (i) is the cruder choice, since it uses only batch sizes and does not account for entry-wise variability across the sub-posterior draws. Scheme (ii) is more appropriate as a practical correction because it down-weights noisier batch contributions element by element, and this is reflected in the closer agreement with the full posterior in both the marginals and the joint plot. Nevertheless, neither scheme is exact here: the sub-posteriors are Wishart rather than Gaussian, and an element-wise weighted average cannot be expected to recover the full joint posterior exactly for moderate batch sizes.

```
def run_part_a(X, batches):
    sub_params = sub_posterior_params(X, batches)
    df_full, scale_full, S = full_posterior_params(X)

    sub_samples = sample_sub_posteriors(sub_params, s=S_DRAW)
    full_samples = wishart.rvs(df=df_full, scale=scale_full, size=S_DRAW,
                               random_state=rng.integers(1e9))
    if full_samples.ndim == 2:
        full_samples = full_samples[np.newaxis]

    si_samples = cmc_scheme_i(sub_samples, batches)
    sii_samples = cmc_scheme_ii(sub_samples)

    pairplot_precision(si_samples, sii_samples, full_samples, OUTPUT_DIR)

    _, cols = upper_tri_samples(full_samples)
    compare_moments(si_samples, sii_samples, full_samples, cols)

    return si_samples, sii_samples, full_samples


X, batches = load_gene_data(GENE_CSV)
```

```
si_samples, sii_samples, full_samples = run_part_a(X, batches)
```

Listing 6: Part (a): reported code excerpt from src/q2_bayes_precision.py

2.2 Part (b): Subsampling Estimators

Let $z_i \overset{\text{iid}}{\sim} \text{Bernoulli}(\lambda)$, $\lambda \in (0, 1]$, be subsampling indicators. Write $f_i = \log p(\mathbf{x}_i \mid \mathbf{\Lambda})$ for the i-th log-likelihood contribution, so that the full unnormalised log-posterior is $\log \gamma(\mathbf{\Lambda}) = \log p(\mathbf{\Lambda}) + \sum_{i=1}^{n} f_i$. The two proposed estimators are

$$\widehat{\log \gamma}(\mathbf{\Lambda}) = \log p(\mathbf{\Lambda}) + \tfrac{1}{\lambda} \sum_{i=1}^{n} z_i \, f_i, \tag{1}$$

$$\widetilde{\log \gamma}(\mathbf{\Lambda}) = \log p(\mathbf{\Lambda}) + \sum_{i=1}^{n} z_i \, f_i. \tag{2}$$

Distributional form implied by (1). Under the model $\mathbf{x}_i \mid \mathbf{\Lambda} \sim \mathcal{N}_d(\mathbf{0}, \mathbf{\Lambda}^{-1})$, the log-likelihood for observation i is

$$f_i = \tfrac{1}{2} \log |\mathbf{\Lambda}| - \tfrac{1}{2} \, \mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i - \tfrac{d}{2} \log(2\pi). \tag{3}$$

The Wishart prior $\mathcal{W}_d(\nu, \mathbf{I})$ with $\nu = d = 6$ has log-density

$$\log p(\mathbf{\Lambda}) = \tfrac{\nu - d - 1}{2} \log |\mathbf{\Lambda}| - \tfrac{1}{2} \text{tr}(\mathbf{I} \, \mathbf{\Lambda}) + c_0, \tag{4}$$

where c_0 does not depend on $\mathbf{\Lambda}$. Substituting (4) and (3) into (1):

$$\begin{aligned} \widehat{\log \gamma}(\mathbf{\Lambda}) &= \log p(\mathbf{\Lambda}) + \frac{1}{\lambda} \sum_{i=1}^{n} z_i \, f_i \\ &= \Big[\tfrac{\nu - d - 1}{2} \log |\mathbf{\Lambda}| - \tfrac{1}{2} \text{tr}(\mathbf{I} \, \mathbf{\Lambda}) \Big] + \frac{1}{\lambda} \sum_{i=1}^{n} z_i \Big[\tfrac{1}{2} \log |\mathbf{\Lambda}| - \tfrac{1}{2} \, \mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i \Big] + \text{const.} \end{aligned} \tag{5}$$

We now collect separately the terms involving $\log |\mathbf{\Lambda}|$ and those involving $\mathbf{\Lambda}$ inside a trace.

Terms in $\log |\mathbf{\Lambda}|$. From the prior in (4): $\tfrac{\nu - d - 1}{2} \log |\mathbf{\Lambda}|$. From the likelihood sum in (5): each z_i contributes $\tfrac{1}{\lambda} \cdot z_i \cdot \tfrac{1}{2} \log |\mathbf{\Lambda}|$; summing over i and writing $N_z = \sum_{i=1}^{n} z_i$ gives $\tfrac{1}{\lambda} \cdot \tfrac{N_z}{2} \log |\mathbf{\Lambda}| = \tfrac{N_z}{2\lambda} \log |\mathbf{\Lambda}|$. Adding the two contributions and putting them over a common denominator:

$$\frac{\nu - d - 1}{2} \log |\mathbf{\Lambda}| + \frac{N_z}{2\lambda} \log |\mathbf{\Lambda}| = \frac{\nu - d - 1}{2} \log |\mathbf{\Lambda}| + \frac{N_z / \lambda}{2} \log |\mathbf{\Lambda}| = \frac{\nu - d - 1 + N_z / \lambda}{2} \log |\mathbf{\Lambda}|. \tag{6}$$

Terms in $\text{tr}(\cdot \, \mathbf{\Lambda})$. From the prior in (4): $-\tfrac{1}{2} \text{tr}(\mathbf{I} \, \mathbf{\Lambda})$. From the likelihood sum in (5): each z_i contributes $-\tfrac{1}{\lambda} \cdot z_i \cdot \tfrac{1}{2} \, \mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i$. Now, for any vector $\mathbf{x}_i$ and matrix $\mathbf{\Lambda}$, the scalar $\mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i$ can be rewritten using the cyclic property of the trace as $\mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i = \text{tr}(\mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i) = \text{tr}(\mathbf{x}_i \mathbf{x}_i^\top \mathbf{\Lambda})$, where the first equality holds because a scalar equals its own trace, and the second applies $\text{tr}(ABC) = \text{tr}(CAB)$. Substituting and summing over i:

$$\frac{1}{\lambda} \sum_{i=1}^{n} z_i \cdot \tfrac{1}{2} \, \mathbf{x}_i^\top \mathbf{\Lambda} \, \mathbf{x}_i = \frac{1}{2\lambda} \sum_{i=1}^{n} z_i \, \text{tr}(\mathbf{x}_i \mathbf{x}_i^\top \mathbf{\Lambda}) = \frac{1}{2} \, \text{tr}\Big(\frac{1}{\lambda} \sum_{i=1}^{n} z_i \, \mathbf{x}_i \mathbf{x}_i^\top \mathbf{\Lambda} \Big),$$

where the last step uses linearity of the trace: $\sum_i \text{tr}(A_i) = \text{tr}(\sum_i A_i)$. Writing $S_z = \sum_{i:\, z_i = 1} \mathbf{x}_i \mathbf{x}_i^\top$ (noting that $z_i \in \{0, 1\}$, so $z_i \, \mathbf{x}_i \mathbf{x}_i^\top$ is $\mathbf{x}_i \mathbf{x}_i^\top$ when $z_i = 1$ and $\mathbf{0}$ otherwise), the expression becomes $\tfrac{1}{2} \text{tr}(S_z / \lambda \, \mathbf{\Lambda})$. Adding the prior contribution:

$$-\tfrac{1}{2} \text{tr}(\mathbf{I} \, \mathbf{\Lambda}) - \tfrac{1}{2} \text{tr}(S_z / \lambda \, \mathbf{\Lambda}) = -\tfrac{1}{2} \text{tr}\big([\mathbf{I} + S_z / \lambda] \, \mathbf{\Lambda} \big), \tag{7}$$

again using linearity of the trace: $\text{tr}(A\mathbf{\Lambda}) + \text{tr}(B\mathbf{\Lambda}) = \text{tr}((A + B)\mathbf{\Lambda})$.

Combining (6) and (7), the estimator (5) takes the form

$$\widehat{\log\gamma}(\mathbf{\Lambda}) = \frac{\nu - d - 1 + N_z/\lambda}{2}\,\log|\mathbf{\Lambda}| \;-\; \frac{1}{2}\,\mathrm{tr}\Big(\big[\mathbf{I} + \tfrac{S_z}{\lambda}\big]\,\mathbf{\Lambda}\Big) + \text{const.} \tag{8}$$

Recall that the Wishart density $\mathcal{W}_d(\delta, \Psi^{-1})$ has log-kernel

$$\frac{\delta - d - 1}{2}\,\log|\mathbf{\Lambda}| \;-\; \frac{1}{2}\,\mathrm{tr}(\Psi\,\mathbf{\Lambda}). \tag{9}$$

Comparing (8) with (9) term by term:

- coefficient of $\log|\mathbf{\Lambda}|$: $\dfrac{\delta - d - 1}{2} = \dfrac{\nu - d - 1 + N_z/\lambda}{2}$; adding $d+1$ to both sides of $\delta - d - 1 = \nu - d - 1 + N_z/\lambda$ gives $\delta = \nu + N_z/\lambda$;
- argument of trace: $\Psi = \mathbf{I} + S_z/\lambda$, so the scale matrix is $\Psi^{-1} = (\mathbf{I} + S_z/\lambda)^{-1}$.

It follows that the distribution $\hat{p}(\mathbf{\Lambda}) \propto \exp\{\widehat{\log\gamma}(\mathbf{\Lambda})\}$ implied by the corrected estimator (1) is

$$\hat{p}(\mathbf{\Lambda} \mid \mathbf{z}, \mathbf{X}) \;=\; \mathcal{W}_d\Big(\nu + \frac{N_z}{\lambda},\;\; \Big(\mathbf{I} + \frac{S_z}{\lambda}\Big)^{-1}\Big). \tag{10}$$

Since each z_i has expectation $\mathbb{E}[z_i] = \lambda$, the expected number of included observations is $\mathbb{E}[N_z] = \mathbb{E}\big[\sum_{i=1}^n z_i\big] = \sum_{i=1}^n \mathbb{E}[z_i] = n\lambda$, so $\mathbb{E}[N_z/\lambda] = n\lambda/\lambda = n$. Similarly, $\mathbb{E}[S_z] = \mathbb{E}\big[\sum_{i=1}^n z_i\,\mathbf{x}_i\mathbf{x}_i^\top\big] = \sum_{i=1}^n \mathbb{E}[z_i]\,\mathbf{x}_i\mathbf{x}_i^\top = \lambda\sum_{i=1}^n \mathbf{x}_i\mathbf{x}_i^\top = \lambda\,S$, so $\mathbb{E}[S_z/\lambda] = \lambda S/\lambda = S$. Hence (10) is centred on the full posterior $\mathcal{W}_d(\nu + n, (\mathbf{I} + S)^{-1})$ in expectation.

Distributional form implied by (2). The only difference between (2) and (1) is that the $1/\lambda$ scaling is absent. Repeating the collection of terms above without this factor:

- the $\log|\mathbf{\Lambda}|$ coefficient from the likelihood sum becomes $\frac{N_z}{2}$ instead of $\frac{N_z}{2\lambda}$, so the total is $\frac{\nu-d-1}{2} + \frac{N_z}{2} = \frac{\nu-d-1+N_z}{2}$;
- the trace argument becomes $\mathbf{I} + S_z$ instead of $\mathbf{I} + S_z/\lambda$.

Matching against the Wishart kernel (9): $\delta - d - 1 = \nu - d - 1 + N_z$, so $\delta = \nu + N_z$; $\Psi = \mathbf{I} + S_z$. It follows that the distribution $\tilde{p}(\mathbf{\Lambda}) \propto \exp\{\widetilde{\log\gamma}(\mathbf{\Lambda})\}$ implied by the naive estimator (2) is

$$\tilde{p}(\mathbf{\Lambda} \mid \mathbf{z}, \mathbf{X}) \;=\; \mathcal{W}_d\big(\nu + N_z,\; (\mathbf{I} + S_z)^{-1}\big). \tag{11}$$

Since $\mathbb{E}[N_z] = n\lambda$ and $\lambda < 1$, we have $\mathbb{E}[N_z] = n\lambda < n$: the effective degrees of freedom are smaller than the full posterior's $\nu + n$, so $\tilde{p}$ is systematically under-concentrated.

Mean, variance, and MSE of $\widehat{\log\gamma}$. Since $z_i \sim \text{Bernoulli}(\lambda)$, its expectation is $\mathbb{E}(z_i) = \lambda$. The only random quantities in (1) are the z_i (the f_i are fixed given $\mathbf{\Lambda}$ and the data), so the expectation of (1) with respect to $\mathbf{z}$ is

$$\mathbb{E}_{\mathbf{z}}\big[\widehat{\log\gamma}\big] = \log p(\mathbf{\Lambda}) + \frac{1}{\lambda}\sum_{i=1}^n \mathbb{E}(z_i)\,f_i = \log p(\mathbf{\Lambda}) + \frac{1}{\lambda}\sum_{i=1}^n \lambda\,f_i = \log p(\mathbf{\Lambda}) + \frac{\lambda}{\lambda}\sum_{i=1}^n f_i = \log p(\mathbf{\Lambda}) + \sum_{i=1}^n f_i = \log\gamma(\mathbf{\Lambda}),$$

where the third equality uses $\lambda/\lambda = 1$. This shows that $\widehat{\log\gamma}$ is unbiased for $\log\gamma(\mathbf{\Lambda})$; hence $\mathrm{Bias}\big[\widehat{\log\gamma}\big] = \mathbb{E}[\widehat{\log\gamma}] - \log\gamma = 0$.

For the variance, note that $\log p(\mathbf{\Lambda})$ is a constant with respect to $\mathbf{z}$, so $\mathrm{Var}_{\mathbf{z}}[\widehat{\log\gamma}] = \mathrm{Var}_{\mathbf{z}}\big(\frac{1}{\lambda}\sum_{i=1}^n z_i\,f_i\big)$. Since $z_1, \ldots, z_n$ are mutually independent, the variance of a sum equals the sum of variances. Furthermore,

each f_i is a constant with respect to $\mathbf{z}$, so $\mathrm{Var}(z_i\, f_i) = f_i^2\, \mathrm{Var}(z_i)$ (using $\mathrm{Var}(cX) = c^2\mathrm{Var}(X)$ for any constant c). With $\mathrm{Var}(z_i) = \lambda(1-\lambda)$:

$$\begin{aligned}
\mathrm{Var}_{\mathbf{z}}\big[\widehat{\log\gamma}\big] &= \frac{1}{\lambda^2}\sum_{i=1}^{n}\mathrm{Var}(z_i\, f_i) && (\mathrm{Var}(aX) = a^2\mathrm{Var}(X) \text{ with } a = 1/\lambda, \text{ then}\\
&= \frac{1}{\lambda^2}\sum_{i=1}^{n} f_i^2\,\mathrm{Var}(z_i) && (\mathrm{Var}(cX) = c^2\mathrm{Var}(X) \text{ with } c = f_i)\\
&= \frac{1}{\lambda^2}\sum_{i=1}^{n} f_i^2\;\lambda(1-\lambda) = \frac{1}{\lambda^2}\;\lambda(1-\lambda)\sum_{i=1}^{n} f_i^2 = \frac{1-\lambda}{\lambda}\sum_{i=1}^{n} f_i^2,
\end{aligned}$$

where the last equality cancels one factor of λ between numerator and denominator: $\lambda(1-\lambda)/\lambda^2 = (1-\lambda)/\lambda$.

Since the estimator is unbiased, $\mathrm{MSE} = \mathrm{Bias}^2 + \mathrm{Var} = 0^2 + \mathrm{Var}$, hence

$$\mathrm{MSE}\big[\widehat{\log\gamma}\big] = \frac{1-\lambda}{\lambda}\sum_{i=1}^{n} f_i^2.$$

Mean, variance, and MSE of $\widetilde{\log\gamma}$. By the same argument, $\mathbb{E}(z_i) = \lambda$. The expectation of (2) is

$$\mathbb{E}_{\mathbf{z}}\big[\widetilde{\log\gamma}\big] = \log p(\mathbf{\Lambda}) + \sum_{i=1}^{n}\mathbb{E}(z_i)\, f_i = \log p(\mathbf{\Lambda}) + \sum_{i=1}^{n}\lambda\, f_i = \log p(\mathbf{\Lambda}) + \lambda\sum_{i=1}^{n} f_i.$$

Since $\lambda \neq 1$, this differs from $\log\gamma(\mathbf{\Lambda}) = \log p(\mathbf{\Lambda}) + \sum_i f_i$, so $\widetilde{\log\gamma}$ is biased. The bias is computed by subtracting:

$$\mathrm{Bias}\big[\widetilde{\log\gamma}\big] = \mathbb{E}\big[\widetilde{\log\gamma}\big] - \log\gamma(\mathbf{\Lambda}) = \Big[\log p(\mathbf{\Lambda}) + \lambda\sum_{i=1}^{n} f_i\Big] - \Big[\log p(\mathbf{\Lambda}) + \sum_{i=1}^{n} f_i\Big] = \lambda\sum_{i=1}^{n} f_i - \sum_{i=1}^{n} f_i = (\lambda-1)\sum_{i=1}^{n} f_i.$$

For the variance: since (2) does *not* have the $1/\lambda$ factor, the variance is simply $\mathrm{Var}_{\mathbf{z}}(\sum_i z_i f_i)$ (without the $1/\lambda^2$ in front). By the same independence and constant-scaling arguments as before:

$$\begin{aligned}
\mathrm{Var}_{\mathbf{z}}\big[\widetilde{\log\gamma}\big] &= \sum_{i=1}^{n}\mathrm{Var}(z_i\, f_i) && (\text{independence of } z_1, \ldots, z_n)\\
&= \sum_{i=1}^{n} f_i^2\,\mathrm{Var}(z_i) && (\mathrm{Var}(cX) = c^2\mathrm{Var}(X) \text{ with } c = f_i)\\
&= \sum_{i=1}^{n} f_i^2\;\lambda(1-\lambda) = \lambda(1-\lambda)\sum_{i=1}^{n} f_i^2.
\end{aligned}$$

Applying $\mathrm{MSE} = \mathrm{Bias}^2 + \mathrm{Var}$:

$$\begin{aligned}
\mathrm{MSE}\big[\widetilde{\log\gamma}\big] &= \Big[(\lambda-1)\sum_{i=1}^{n} f_i\Big]^2 + \lambda(1-\lambda)\sum_{i=1}^{n} f_i^2\\
&= (\lambda-1)^2\Big(\sum_{i=1}^{n} f_i\Big)^2 + \lambda(1-\lambda)\sum_{i=1}^{n} f_i^2 && ((ab)^2 = a^2b^2)\\
&= (1-\lambda)^2\Big(\sum_{i=1}^{n} f_i\Big)^2 + \lambda(1-\lambda)\sum_{i=1}^{n} f_i^2, && ((\lambda-1)^2 = (1-\lambda)^2)
\end{aligned}$$

where the last equality uses $(\lambda-1)^2 = (-(1-\lambda))^2 = (1-\lambda)^2$.

The results are collected in Table 3. These derivations follow Ch. 8, §8.2 of the lecture notes.

	Corrected $\big(\widehat{\log\gamma}\big)$	Naive $\big(\widetilde{\log\gamma}\big)$
Implied form	$\mathcal{W}_d\big(\nu+\frac{N_z}{\lambda},\,(\mathbf{I}+\frac{S_z}{\lambda})^{-1}\big)$	$\mathcal{W}_d\big(\nu+N_z,\,(\mathbf{I}+S_z)^{-1}\big)$
Bias	0	$(\lambda-1)\sum_i f_i$
Variance	$\dfrac{1-\lambda}{\lambda}\sum_i f_i^2$	$\lambda(1-\lambda)\sum_i f_i^2$
MSE	$\dfrac{1-\lambda}{\lambda}\sum_i f_i^2$	$\lambda(1-\lambda)\sum_i f_i^2 + (1-\lambda)^2\big(\sum_i f_i\big)^2$

Table 3: Subsampling estimators of $\log\gamma(\mathbf{\Lambda})$: implied Wishart form, bias, variance, and MSE.

2.3 Part (c): Inflated Batch-Specific Distribution

Consider the inflated sub-posterior

$$\tilde{p}(\mathbf{\Lambda} \mid \mathbf{x}_b) \propto p(\mathbf{\Lambda})\, p(\mathbf{x}_b \mid \mathbf{\Lambda})^{n/n_b} = p(\mathbf{\Lambda})\, \exp\Big(\tfrac{n}{n_b}\, \log p(\mathbf{x}_b \mid \mathbf{\Lambda})\Big).$$

Since

$$\log p(\mathbf{x}_b \mid \mathbf{\Lambda}) = \sum_{i\in b} \log p(x_i \mid \mathbf{\Lambda}),$$

we obtain

$$\tfrac{n}{n_b}\, \log p(\mathbf{x}_b \mid \mathbf{\Lambda}) = \sum_{i\in b} \tfrac{n}{n_b}\, \log p(x_i \mid \mathbf{\Lambda}),$$

so each observation in batch b is effectively weighted by n/n_b. This is equivalent to replicating the batch up to total size n, so that machine b behaves as if it had observed n data points.

Consensus Monte Carlo is valid only if

$$\prod_{b=1}^{B} \tilde{p}(\mathbf{\Lambda} \mid \mathbf{x}_b) \;\propto\; p(\mathbf{\Lambda})\, p(\mathbf{X} \mid \mathbf{\Lambda}).$$

Here instead

$$\prod_{b=1}^{B} \tilde{p}(\mathbf{\Lambda} \mid \mathbf{x}_b) \;\propto\; p(\mathbf{\Lambda})^B \prod_{b=1}^{B} p(\mathbf{x}_b \mid \mathbf{\Lambda})^{n/n_b}.$$

Thus the prior is counted B times and each batch likelihood is over-weighted by n/n_b, so the combined distribution is overly concentrated and does not match $p(\mathbf{\Lambda} \mid \mathbf{X})$. Therefore CMC is not appropriate for approximating $p(\mathbf{\Lambda} \mid \mathbf{X})$.

The alternatives are necessarily heuristic, as the inflated sub-posteriors do not form a valid factorisation of the full posterior, and therefore no combination rule can recover $p(\mathbf{\Lambda} \mid \mathbf{X})$ exactly. One option is to retain a single inflated sub-posterior, preferably the one from the largest batch,

$$b^* = \arg\max_b n_b,$$

since its inflation factor

$$\frac{n}{n_{b^*}} = \min_b \frac{n}{n_b}$$

is the smallest. In that case,

$$\tilde{p}(\mathbf{\Lambda} \mid \mathbf{x}_{b^*}) \propto p(\mathbf{\Lambda})\, p(\mathbf{x}_{b^*} \mid \mathbf{\Lambda})^{n/n_{b^*}}$$

is the least distorted among the inflated sub-posteriors, although it still depends only on the local batch and therefore does not equal the full posterior.

Another possibility is to combine only posterior summaries rather than the full sample clouds. For example, if

$$\hat{\mathbf{\Lambda}}_b = \mathbb{E}_{\tilde{p}(\mathbf{\Lambda}|\mathbf{x}_b)}[\mathbf{\Lambda}],$$

one may form the average

$$\bar{\mathbf{\Lambda}} = \frac{1}{B}\sum_{b=1}^{B}\hat{\mathbf{\Lambda}}_b.$$

This avoids multiplying the inflated sub-posteriors and hence avoids the over-counting problem, but it only produces a point estimate and discards posterior uncertainty. Therefore these strategies may yield rough approximations, but they do not recover the exact full posterior.

2.4 Part (d): Distributed Gradient Descent with Doubly Stochastic W

We first complete the weight matrix $\mathbf{W}$ by imposing that it is doubly stochastic, i.e. all entries are non-negative and each row and column sums to 1. From the given rows,

$$W_{11} + W_{12} = \tfrac{1}{2}, \qquad W_{21} + W_{22} = \tfrac{1}{2},$$

and from the first two column sums,

$$W_{11} + W_{21} = \tfrac{1}{2}, \qquad W_{12} + W_{22} = \tfrac{1}{2}.$$

By symmetry this gives

$$W_{11} = W_{12} = W_{21} = W_{22} = \tfrac{1}{4}.$$

Similarly, using the row and column sums for rows 4–5 and columns 3–5 gives

$$W_{43} = 0, \quad W_{44} = \tfrac{1}{4}, \quad W_{45} = \tfrac{1}{4}, \qquad W_{53} = 0, \quad W_{54} = 0, \quad W_{55} = \tfrac{1}{2}.$$

Hence

$$\mathbf{W} = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & 0 & 0 & \frac{1}{2} \end{pmatrix}.$$

For $x_i \sim N_d(0, \mathbf{\Lambda}^{-1})$, the negative log-likelihood based on batch b is, up to an additive constant,

$$f_b(\mathbf{\Lambda}) = -\frac{n_b}{2}\log|\mathbf{\Lambda}| + \frac{1}{2}\sum_{i\in b} x_i^\top \mathbf{\Lambda} x_i = -\frac{n_b}{2}\log|\mathbf{\Lambda}| + \frac{1}{2}\mathrm{tr}(S_b\mathbf{\Lambda}),$$

where

$$S_b = \sum_{i\in b} x_i x_i^\top.$$

Using

$$\nabla_{\mathbf{\Lambda}}\log|\mathbf{\Lambda}| = \mathbf{\Lambda}^{-1}, \qquad \nabla_{\mathbf{\Lambda}}\mathrm{tr}(S_b\mathbf{\Lambda}) = S_b,$$

the local gradient is

$$\nabla f_b(\mathbf{\Lambda}) = -\frac{n_b}{2}\mathbf{\Lambda}^{-1} + \frac{1}{2}S_b.$$

If agent b uses a mini-batch $\mathcal{I}_b^{(t)}$ of size m_b, an unbiased stochastic approximation is

$$\widehat{S}_b^{(t)} = \frac{n_b}{m_b}\sum_{i\in\mathcal{I}_b^{(t)}} x_i x_i^\top, \qquad \widehat{\nabla} f_b(\mathbf{\Lambda}_b^{(t)}) = -\frac{n_b}{2}(\mathbf{\Lambda}_b^{(t)})^{-1} + \frac{1}{2}\widehat{S}_b^{(t)}.$$

The unbiasedness is immediate when the mini-batch is sampled uniformly from batch b:

$$\mathbb{E}\left[\frac{1}{m_b}\sum_{i\in\mathcal{I}_b^{(t)}} x_i x_i^\top\right] = \frac{1}{n_b}\sum_{i\in b} x_i x_i^\top = \frac{S_b}{n_b},$$

so multiplying by n_b gives

$$\mathbb{E}[\widehat{S}_b^{(t)}] = \mathbb{E}\left[\frac{n_b}{m_b}\sum_{i\in\mathcal{I}_b^{(t)}} x_i x_i^\top\right] = S_b.$$

Therefore

$$\mathbb{E}\big[\widehat{\nabla} f_b(\mathbf{\Lambda}_b^{(t)})\big] = -\frac{n_b}{2}(\mathbf{\Lambda}_b^{(t)})^{-1} + \frac{1}{2}\mathbb{E}[\widehat{S}_b^{(t)}] = \nabla f_b(\mathbf{\Lambda}_b^{(t)}),$$

so the mini-batch gradient is unbiased for the full local gradient.

The distributed gradient descent update is therefore

$$\mathbf{\Lambda}_b^{(t+1)} = \sum_{c=1}^{B} W_{bc}\Big(\mathbf{\Lambda}_c^{(t)} - \eta_t\,\widehat{\nabla} f_c(\mathbf{\Lambda}_c^{(t)})\Big), \qquad b = 1,\dots,B.$$

This is a *synchronous* algorithm, since at iteration $t+1$ each agent uses the current iterates/gradients from the same round t before applying the consensus step.

3 Question 3: Markov Mixture Model, Variational Inference, and EWMA [25 marks]

3.1 Part (a): Sufficient Statistics

We are asked to derive a minimal and complete sufficient statistic for ϕ_0 and $\phi_{k,m}$ when the labels z are treated as known.

Assume the labels $z = (z_1, \ldots, z_n)$ are known. Since z is observed, the mixture structure disappears and the likelihood decomposes into independent Markov chains indexed by k.

Conditional on z and the parameters, the trajectories are independent across i, as implied by the model specification. Moreover, each sequence satisfies the first-order Markov property, so for sequence i,

$$p(x_i \mid z_i, \phi_0, \{\phi_{k,m}\}) = \phi_{0,x_{i,1}} \prod_{j=2}^{L_i} \phi_{z_i,x_{i,j-1},x_{i,j}}.$$

Hence the likelihood factorises as

$$p(x_{1:n} \mid z, \phi_0, \{\phi_{k,m}\}) = \prod_{i=1}^{n} \left(\phi_{0,x_{i,1}} \prod_{j=2}^{L_i} \phi_{z_i,x_{i,j-1},x_{i,j}} \right).$$

Define the initial-state counts

$$T_{0,\ell} = \sum_{i=1}^{n} \mathbf{1}(x_{i,1} = \ell),$$

and the labelled transition counts

$$T_{k,m,\ell} = \sum_{i=1}^{n} \mathbf{1}(z_i = k) \sum_{j=2}^{L_i} \mathbf{1}(x_{i,j-1} = m,\ x_{i,j} = \ell).$$

Then the likelihood can be written as

$$p(x_{1:n} \mid z, \phi_0, \{\phi_{k,m}\}) = \prod_{\ell=1}^{M} \phi_{0,\ell}^{T_{0,\ell}} \prod_{k=1}^{K} \prod_{m=1}^{M} \prod_{\ell=1}^{M} \phi_{k,m,\ell}^{T_{k,m,\ell}}.$$

Thus, the likelihood depends on $(\phi_0, \{\phi_{k,m}\})$ only through

$$T = \Big((T_{0,\ell})_{\ell=1}^{M},\ (T_{k,m,\ell})_{k,m,\ell} \Big),$$

so by the Fisher–Neyman factorisation theorem, T is sufficient.

Moreover, each block corresponds to a categorical (or multinomial) model: one for ϕ_0 and, for each (k, m), one for $\phi_{k,m}$. The associated count vectors are the natural sufficient statistics; no further reduction is possible since different datasets with the same counts yield the same likelihood, so the statistic is minimal.

Since these models form a full exponential family, the sufficient statistic is also complete.

3.2 Part (b): Coordinate Ascent Variational Inference

We approximate the posterior with the mean-field family

$$q(\mathbf{z}, \Phi) = \prod_{i=1}^{n} q_{z_i}(z_i)\ q_{\boldsymbol{\phi}_0}(\boldsymbol{\phi}_0) \prod_{k=1}^{K} \prod_{m=1}^{M} q_{\boldsymbol{\phi}_{k,m}}(\boldsymbol{\phi}_{k,m}),$$

where $q_{z_i}(z_i) = \mathrm{Cat}(\tilde{\boldsymbol{\pi}}_i)$ and all $\boldsymbol{\phi}$-factors are Dirichlet. The prior $z_i \sim \mathrm{Cat}(1/K, \ldots, 1/K)$ is fixed, hence no variational factor for mixing weights appears.

General CAVI update. For each factor,

$$\log q_j^*(\theta_j) = \mathbb{E}_{q_{-j}}[\log p(\mathbf{X}, \mathbf{z}, \Phi)] + \text{const.}$$

Update for $q^*(z_i)$. For fixed i, the terms depending on z_i are

$$\log p(z_i) + \log \phi_{0,x_{i,1}} + \sum_{t=2}^{L_i} \log \phi_{z_i,x_{i,t-1},x_{i,t}}.$$

Taking expectations,

$$\log q^*(z_i = k) = \log(1/K) + \mathbb{E}[\log \phi_{0,x_{i,1}}] + \sum_{t=2}^{L_i} \mathbb{E}[\log \phi_{k,x_{i,t-1},x_{i,t}}] + c.$$

The first two terms are constant in k and are absorbed into c, hence

$$\log q^*(z_i = k) = \sum_{t=2}^{L_i} \mathbb{E}[\log \phi_{k,x_{i,t-1},x_{i,t}}] + c.$$

Using the Dirichlet expectation,

$$\mathbb{E}[\log \phi_{k,m,m'}] = \psi(\rho_{k,m,m'}) - \psi\Big(\sum_{m''} \rho_{k,m,m''}\Big),$$

and rewriting via transition counts $N_{i,m,m'}$,

$$\log q^*(z_i = k) = \sum_{m,m'} N_{i,m,m'} \Big[\psi(\rho_{k,m,m'}) - \psi\big(\sum_{m''} \rho_{k,m,m''}\big)\Big] + c.$$

Exponentiating,

$$q^*(z_i = k) \propto \exp(\eta_{i,k}), \quad \eta_{i,k} = \sum_{m,m'} N_{i,m,m'} \Big[\psi(\rho_{k,m,m'}) - \psi\big(\sum_{m''} \rho_{k,m,m''}\big)\Big],$$

and normalising,

$$\tilde{\pi}_{i,k} = \frac{\exp(\eta_{i,k})}{\sum_{k'} \exp(\eta_{i,k'})}.$$

Update for $q^*(\boldsymbol{\phi}_0)$. The terms involving $\boldsymbol{\phi}_0$ are

$$(\alpha - 1) \sum_m \log \phi_{0,m} + \sum_i \log \phi_{0,x_{i,1}}.$$

Grouping by state,

$$\log q^*(\boldsymbol{\phi}_0) = \sum_{m=1}^{M} (\alpha - 1 + T_{0,m}) \log \phi_{0,m} + c,$$

hence

$$q^*(\boldsymbol{\phi}_0) = \text{Dir}(\boldsymbol{\rho}_0), \qquad \rho_{0,m} = \alpha + T_{0,m}.$$

Update for $q^*(\boldsymbol{\phi}_{k,m})$. The terms involving $\boldsymbol{\phi}_{k,m}$ are

$$(\alpha - 1) \sum_{m'} \log \phi_{k,m,m'} + \sum_i \tilde{\pi}_{i,k} \sum_{t \geq 2} \mathbf{1}[x_{i,t-1} = m] \log \phi_{k,m,x_{i,t}}.$$

Rewriting using counts,

$$\log q^*(\boldsymbol{\phi}_{k,m}) = \sum_{m'=1}^{M} \Big(\alpha - 1 + \sum_i \tilde{\pi}_{i,k} N_{i,m,m'}\Big) \log \phi_{k,m,m'} + c,$$

hence

$$q^*(\boldsymbol{\phi}_{k,m}) = \text{Dir}(\boldsymbol{\rho}_{k,m}), \qquad \rho_{k,m,m'} = \alpha + \sum_i \tilde{\pi}_{i,k} N_{i,m,m'}.$$

Form of the optimal variational approximation. Collecting the updates,

$$q^*(\mathbf{z}, \Phi) = \prod_{i=1}^{n} \mathrm{Cat}(z_i; \tilde{\boldsymbol{\pi}}_i)\ \mathrm{Dir}(\boldsymbol{\phi}_0; \boldsymbol{\rho}_0)\ \prod_{k=1}^{K} \prod_{m=1}^{M} \mathrm{Dir}(\boldsymbol{\phi}_{k,m}; \boldsymbol{\rho}_{k,m}),$$

with parameters given above.

ELBO decomposition. The evidence lower bound is

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(\mathbf{X}, \mathbf{z}, \Phi)] - \mathbb{E}_q[\log q(\mathbf{z}, \Phi)].$$

Under the mean-field factorisation,

$$\mathcal{L}(q) = \underbrace{\mathbb{E}_q[\log p(\mathbf{X}, \mathbf{z} \mid \Phi)]}_{\text{data term}} \underbrace{- \mathrm{KL}(q(\boldsymbol{\phi}_0) \| p(\boldsymbol{\phi}_0)) - \sum_{k,m} \mathrm{KL}(q(\boldsymbol{\phi}_{k,m}) \| p(\boldsymbol{\phi}_{k,m}))}_{\text{Dirichlet KL}} + \underbrace{\sum_i H(q(z_i))}_{\text{entropy}},$$

where

$$H(q(z_i)) = -\sum_k \tilde{\pi}_{i,k} \log \tilde{\pi}_{i,k}.$$

Expanding the data term,

$$\mathbb{E}_q[\log p(\mathbf{X}, \mathbf{z} \mid \Phi)] = \sum_i \sum_k \tilde{\pi}_{i,k} \log(1/K) + \sum_i \mathbb{E}_q[\log \phi_{0,x_{i,1}}] + \sum_i \sum_k \tilde{\pi}_{i,k} \sum_{m,m'} N_{i,m,m'}\, \mathbb{E}_q[\log \phi_{k,m,m'}].$$

The CAVI algorithm alternates these updates while $\mathcal{L}(q)$ is non-decreasing. The implementation used for this part is a direct NumPy/SciPy realisation of these updates: trajectories are first converted into initial-state and transition-count arrays, then the responsibilities $\tilde{\pi}_{i,k}$ are updated with digamma expectations, the Dirichlet parameters $\rho_{0,m}$ and $\rho_{k,m,m'}$ are refreshed from the current soft counts, and the ELBO is evaluated at each iteration to monitor convergence. Full code is given in Appendix B.3. Since Part (b) is a derivation question, the appendix code includes a small synthetic verification of the closed-form updates rather than a separate data-analysis task on Athena.

3.3 Part (c): EWMA Changepoint Detection

We construct an EWMA control chart for the stream of initial positions $x_{1,1}, \ldots, x_{n,1}$ to detect a change in $\boldsymbol{\phi}_0$.

Each observation $x_{i,1} \in \{1, \ldots, M\}$ is encoded as a one-hot vector $e_t \in \{0, 1\}^M$. Under H_0, e_t is i.i.d. with mean $\boldsymbol{\mu}$ and covariance

$$\boldsymbol{\Sigma} = \mathrm{diag}(\boldsymbol{\mu}) - \boldsymbol{\mu}\boldsymbol{\mu}^\top,$$

which is singular. Although $M = 16$, the supplied in-control mean $\boldsymbol{\mu}$ has positive mass on only six states, so

$$\mathrm{rank}(\boldsymbol{\Sigma}) = 6 - 1 = 5.$$

We define

$$Z_t = (1 - \rho) Z_{t-1} + \rho e_t, \quad Z_0 = \boldsymbol{\mu},$$

with $\rho = 0.1$. Unrolling the recursion,

$$Z_t - \boldsymbol{\mu} = \rho \sum_{j=0}^{t-1} (1 - \rho)^j (e_{t-j} - \boldsymbol{\mu}),$$

so that

$$\boldsymbol{\Sigma}_{Z_t} = \mathrm{Var}(Z_t) = \frac{\rho}{2 - \rho} \big[1 - (1 - \rho)^{2t}\big] \boldsymbol{\Sigma}.$$

Since $Z_t - \boldsymbol{\mu}$ is vector-valued, we require a scalar statistic to monitor departures from the in-control distribution. Under H_0, by the multivariate central limit theorem,

$$Z_t - \boldsymbol{\mu} \overset{a}{\sim} \mathcal{N}(0, \boldsymbol{\Sigma}_{Z_t}).$$

A natural way to quantify deviations is therefore through a covariance-standardised quadratic form, leading to

$$S_t = (Z_t - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}_{Z_t}^{+} (Z_t - \boldsymbol{\mu}),$$

where $^+$ denotes the Moore–Penrose pseudoinverse[1].

Since this quadratic form is (asymptotically) a sum of squared Gaussian components, it follows that

$$S_t \overset{a}{\sim} \chi^2_r, \quad r = \operatorname{rank}(\boldsymbol{\Sigma}) = 5.$$

We signal a changepoint when S_t exceeds a threshold L, i.e.

$$A_t = \mathbf{1}\{S_t > L\}, \quad t^* = \min\{t : S_t > L\}.$$

For a target false-alarm level α, we set

$$L_\alpha = \chi^2_{r,1-\alpha}.$$

In particular, with $r = 5$ and $\alpha = 0.001$[2], we obtain

$$L = \chi^2_{5,0.999} = 20.52.$$

Implementation (core logic). The core Python implementation mirrors these definitions; the full code is provided in Appendix B.3. The essential logic is:

```
Z = mu.copy()
S = np.zeros(n)

for t in range(1, n):
    fac = rho / (2 - rho) * (1 - (1 - rho)**(2*t))
    Z = (1 - rho) * Z + rho * E[t]
    d = Z - mu
    S[t] = float(d @ (np.linalg.pinv(Sigma) / fac) @ d)

cp = int(np.where(S > L)[0][0])
```

Listing 7: Q3(c): EWMA core logic

Result. For $\rho = 0.1$, the statistic first exceeds L at

$$t^* = 258.$$

Hence a changepoint is detected.

Possible issues.

1. **Asymptotic χ^2 approximation.** The χ^2 approximation relies on asymptotic normality of $Z_t - \boldsymbol{\mu}$, which may be inaccurate for small t or highly discrete multinomial data.
2. **Singular covariance.** Since $\boldsymbol{\Sigma}$ is singular, the use of the pseudoinverse $\boldsymbol{\Sigma}^+$ may lead to numerical instability and sensitivity to small perturbations.
3. **Misspecification of $\boldsymbol{\mu}$.** If the in-control mean $\boldsymbol{\mu}$ is estimated or misspecified, the calibration of the control limit is affected, leading to incorrect false-alarm rates.

[1] $\boldsymbol{\Sigma}$ is singular, so it is not invertible. If $A = U \operatorname{diag}(\sigma_i) V^\top$, then $A^+ = V \operatorname{diag}(\sigma_i^{-1} \mathbf{1}_{\{\sigma_i > 0\}}) U^\top$.

[2] We adopt a small significance level $\alpha = 0.001$ to limit the accumulation of false positives in a sequential testing setting, where the monitoring statistic is evaluated repeatedly over time.

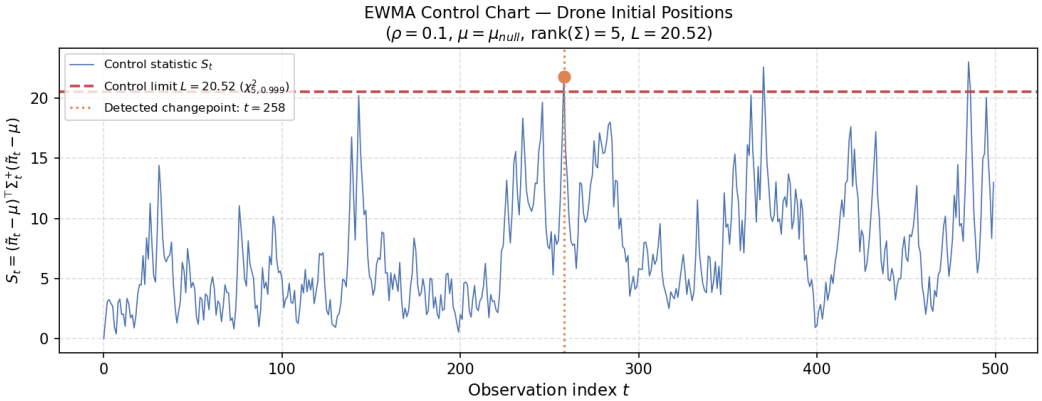


Figure 4: EWMA chart for S_t with control limit L and first exceedance at $t^* = 258$. The threshold line and changepoint marker are shown explicitly.

4. **Temporal dependence.** If the observations are not independent, the variance of Z_t is underestimated, increasing the likelihood of false positives.

5. **Sequential testing.** Since the test is performed repeatedly over time, the overall probability of false alarms is larger than the nominal level α.

Full implementation details are provided in Appendix B.3.

4 Question 4: Gamma-GLM Analysis of EPA Daily AQI [25 marks]

4.1 Dataset and research question

I use the U.S. EPA *Daily AQI by County* public dataset for 2022–2023. After cleaning, this yields 649,818 county-day observations with AQI, date, defining pollutant, and number of reporting sites. The dataset is suitable for this coursework because it is publicly available, naturally tabular, and observed repeatedly over counties and dates, with enough scale to make grouped aggregations, county-wise windows, and repeated joins non-trivial. The `num_sites` field also exposes heterogeneous monitoring coverage, which is useful both statistically and for discussing data quality. Although this two-year slice is not so large that a single-machine analysis would be impossible, it is large enough that a scalable Spark DataFrame workflow is the right methodological choice, and the same pipeline extends directly to longer horizons without rewriting the analysis.

The scientific question is: *to what extent do seasonality, geography, pollutant identity, and historical county air-quality profile explain variation in daily AQI across U.S. counties, and how much additional predictive power is gained from short-run within-county temporal dependence?*

This is better posed as a regression problem than as a classifier. Although one could instead model the exceedance event $AQI > 100$, only 1.46% of observations satisfy this threshold, so a binary exceedance model would be dominated by class imbalance and would discard most of the information in AQI. I therefore model the continuous AQI response directly. Since Gamma regression requires strict positivity, the 1,098 rows with AQI= 0 are excluded, leaving 648,720 observations for fitting and evaluation.

4.2 Exploratory data analysis

Three empirical patterns motivate the final model. First, AQI is positive and right-skewed: the mean is 43.4, with quartiles 32, 42, and 52, so a Gaussian constant-variance model is not attractive. Second, seasonality is clear: the national monthly mean rises from 40.6 in January to 53.2 in June before falling to 38.1 in December, and the unhealthy-day rate peaks in June at 5.6%. Third, regional differences are not just level shifts: the Midwest has the highest mean AQI (44.0), but the West has the highest unhealthy-day rate (2.37%), indicating a heavier upper tail and stronger episodic summer extremes. Ozone and $PM_{2.5}$ together account for roughly 94.5% of unhealthy observations, which further supports explicit pollutant effects. These features suggest a model with positive support, non-constant variance, and explicit seasonal and geographic terms.

EDA also clarifies what should not be the main analysis. Because unhealthy AQI days are rare, a Bernoulli model would be weakly informative for most of the dataset. Equally, a single national time trend would miss the clear regional differences in the annual AQI cycle. A supplementary state-level choropleth in Appendix A.1 shows that the highest unhealthy-day rates are concentrated in California and a small set of western and midwestern states.

4.3 Statistical model and justification

My main scientific model is a Gamma GLM with log link fitted to the positive AQI sample, using all 2022 observations for training and 2023 as a disjoint test set. Writing Y_{ct} for AQI in county c on day t,

$$Y_{ct} \mid X_{ct} \sim \Gamma(\mu_{ct}, \phi), \qquad \log \mu_{ct} = \beta_0 + X_{ct}^{\top}\beta,$$

where X_{ct} contains seasonal harmonics, Census-division effects, defining-pollutant effects, log number of reporting sites, and three county history features computed from 2022 only: county mean AQI, county summer uplift, and county $PM_{2.5}$ share. I also include pollutant-by-season and region-by-season interactions so that the annual pattern can vary across broad pollutant regimes and geographic areas.

This choice is principled. The Gamma family matches the positive, skewed response and allows the variance to increase with the mean; the log link keeps predictions positive and makes coefficients interpretable as multiplicative AQI effects. The county history features act as parsimonious proxies

Model	Mean Gamma dev.	RMSE	Corr.
Null mean	0.2389	22.70	–
Structural Gamma	0.1767	20.46	0.439
Hybrid Gamma (with lags)	**0.1077**	**16.41**	**0.694**

Table 4: Held-out 2023 comparison on the 2022 → 2023 split. The hybrid model is reported as a predictive upper bound rather than as the main interpretive model.

for persistent county heterogeneity without introducing a more complex hierarchical model that is unnecessary for this coursework.

For predictive benchmarking only, I also fit a hybrid Gamma GLM that augments the same structural design with county-wise lagged AQI at 1, 7, and 30 days and a backward-looking 7-day rolling mean. These features are useful for measuring predictive gain, but they are not the main scientific model because they capture temporal persistence in AQI itself rather than structural drivers.

4.4 PySpark / Athena implementation

All analysis is implemented in PySpark on Athena, accessed remotely via SSH. I use Spark DataFrames throughout rather than RDDs: the EPA files are loaded as DataFrames, cleaned with `withColumn` and `groupBy`, county-level temporal features are created with partitioned window functions, and the GLMs are fitted with `pyspark.ml` using `VectorAssembler` and `GeneralizedLinearRegression`. Spark is not used decoratively here. The computationally relevant steps are exactly the ones emphasized in the module: wide aggregations, county-date joins, and lagged feature construction over many partitions. Expressing them as DataFrames keeps the workflow lazy, parallelised inside Spark's execution engine, and easy to scale beyond the present subset. Relative to a pure local-Python or pandas workflow, Spark distributes the county-wise window operations and joins across memory and executors and scales directly to longer EPA horizons or the full archive. Only compact summaries are collected locally for plotting. The active script is written to inherit Athena's Spark configuration rather than forcing local execution.

Leakage is controlled carefully. All county history summaries are computed from 2022 only and then joined onto both years, while the lagged features for 2023 use only past AQI values within county. Full code is moved to Appendix B.4 and `src/q4_analysis.ipynb`.

4.5 Results

Table 4 reports held-out 2023 performance. Mean Gamma deviance is the primary selection criterion because it matches the assumed response family; RMSE and correlation are included as secondary summaries.

The structural model reduces mean Gamma deviance by 26.0% relative to the null benchmark, so the seasonal, geographic, pollutant, and county-history covariates explain substantial AQI variation while remaining interpretable. The hybrid model improves much further, reducing mean Gamma deviance by 39.1% and RMSE by 19.8% relative to the structural model. This indicates strong short-run within-county persistence beyond what can be captured by coarse exogenous covariates alone. Thus structural variables explain a substantial but incomplete share of AQI variability. A concrete diagnostic confirms the remaining misspecification: in the structural model, observed mean AQI exceeds fitted mean AQI in calibration deciles 0–8, and the monthly diagnostic underestimates the June mean by about 14.2 AQI points, so the summer peak is still too smooth.

4.6 Interpretation

Interpretation is based on the structural model, not the hybrid one. At this sample size, formal significance is not the main issue: even small effects can be estimated very precisely. I therefore focus on effect size, interval estimates, and what they imply scientifically. The strongest structural result is persistence in county baseline air quality. Moving from the 25th to the 75th percentile of historical county mean AQI multiplies expected 2023 AQI by approximately 1.233, even after adjusting for season,

pollutant, and monitoring intensity. This is a practically large effect, so a purely national seasonal explanation is inadequate.

Pollutant identity also matters after this adjustment. Relative to ozone days, $PM_{2.5}$ days have 1.158 times larger expected AQI (95% CI $[1.153, 1.163]$), so the interval excludes 1 by a wide margin and the effect is substantively, not just statistically, meaningful. By contrast, NO_2 and CO days are associated with much lower expected AQI ratios of 0.686 and 0.267. Geographic main effects are more modest once county history is included; for example, West South Central is about 1.048 times New England. The remaining geographic signal is mainly seasonal, consistent with the stronger summer deterioration observed in the West.

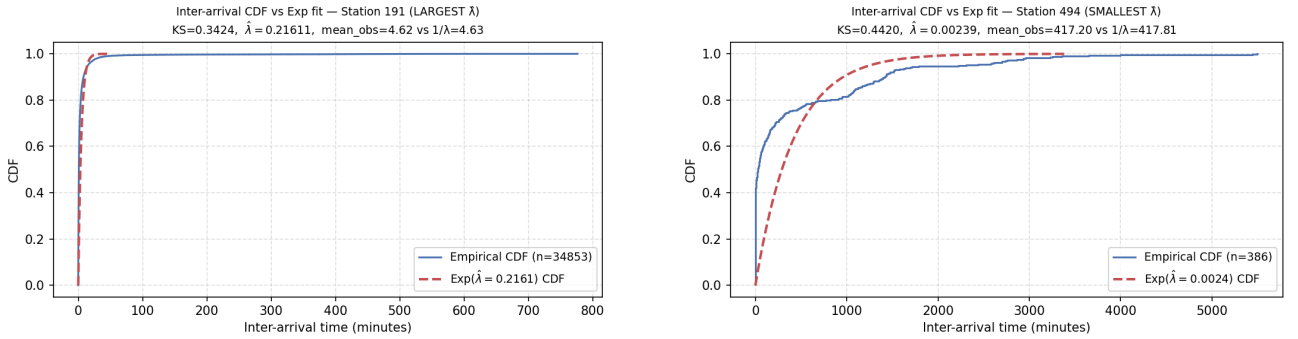
Overall, the analysis suggests that AQI is shaped by persistent county heterogeneity, pollutant composition, and seasonality. The large additional gain from the hybrid model implies that the structural covariates do not absorb all dynamic variation; short-run serial dependence remains, plausibly because weather and smoke transport evolve faster than the coarse annual covariates used here.

4.7 Limitations

The main limitations concern data quality and representativeness as much as model form. AQI is an engineered index rather than a raw pollutant concentration, so the Gamma law is an approximation. More importantly, a large number of county-days does not remove systematic bias: in the "Big Data paradox" sense emphasised in the course, one can estimate the wrong target very precisely if monitoring coverage is uneven. Counties differ sharply in monitor density and siting, urban areas are more intensively observed, and county-days are not a random sample of U.S. exposure conditions. The `num_sites` covariate partially adjusts for this heterogeneity, but it does not make the sample representative. The county history terms are predictive summaries, not causal mechanisms, and the hybrid lag terms improve prediction but should not be interpreted structurally. Weather, wildfire smoke transport, and local emissions remain unmodelled, which explains why even the better-performing models smooth the most extreme summer episodes.

A Q1 Sanity Check: Empirical CDF vs. Exponential Fit

Figure 5 overlays the empirical inter-arrival CDF against the fitted Exp($\hat{\lambda}_i$) for the busiest and quietest stations. At station 191 the departure is most visible in the short-inter-arrival region, consistent with bursty commute-peak arrivals. At station 494 the agreement is visually closer, but with fewer than 400 events the test has limited power.



(a) Station 191 ($\hat{\lambda} = 0.2161$): clearest departure.

(b) Station 494 ($\hat{\lambda} = 0.00239$): apparent fit, low power.

Figure 5: Empirical inter-arrival CDF vs. Exp($\hat{\lambda}_i$) for extreme stations.

```python
def sanity_check_interarrivals(bikes):
    extremes = (bikes.select("start_id", "lambda_hat")
                .dropDuplicates(["start_id"]))
    largest  = extremes.orderBy(F.desc("lambda_hat")).first()["start_id"]
    smallest = extremes.orderBy(F.asc("lambda_hat")).first()["start_id"]

    for station_id in [largest, smallest]:
        rows = (bikes.filter(F.col("start_id") == station_id)
                .select("inter_arrival", "lambda_hat").toPandas())
        lam = float(rows["lambda_hat"].iloc[0])
        ia  = rows["inter_arrival"].values
        ia_sorted = np.sort(ia)
        n_i = len(ia)
        ecdf = np.arange(1, n_i + 1) / n_i
        x_grid = np.linspace(0, np.percentile(ia_sorted, 99), 300)
        exp_cdf = 1 - np.exp(-lam * x_grid)
        ks_stat = np.max(np.abs(ecdf - (1 - np.exp(-lam * ia_sorted))))
        # Plot empirical CDF vs Exp CDF (see source for full plotting code)
```

Listing 8: Q1 sanity check: ECDF vs. exponential fit (from `src/q1_window_functions.py`, lines 258–334)

A.1 Question 4 Supplementary Map

Figure 6 gives a supplementary geographic diagnostic for Question 4. It maps the state-level share of county-days with AQI> 100 over 2022–2023. This is not used directly in the model, but it helps visualise the spatial concentration of unhealthy episodes discussed in the main text.

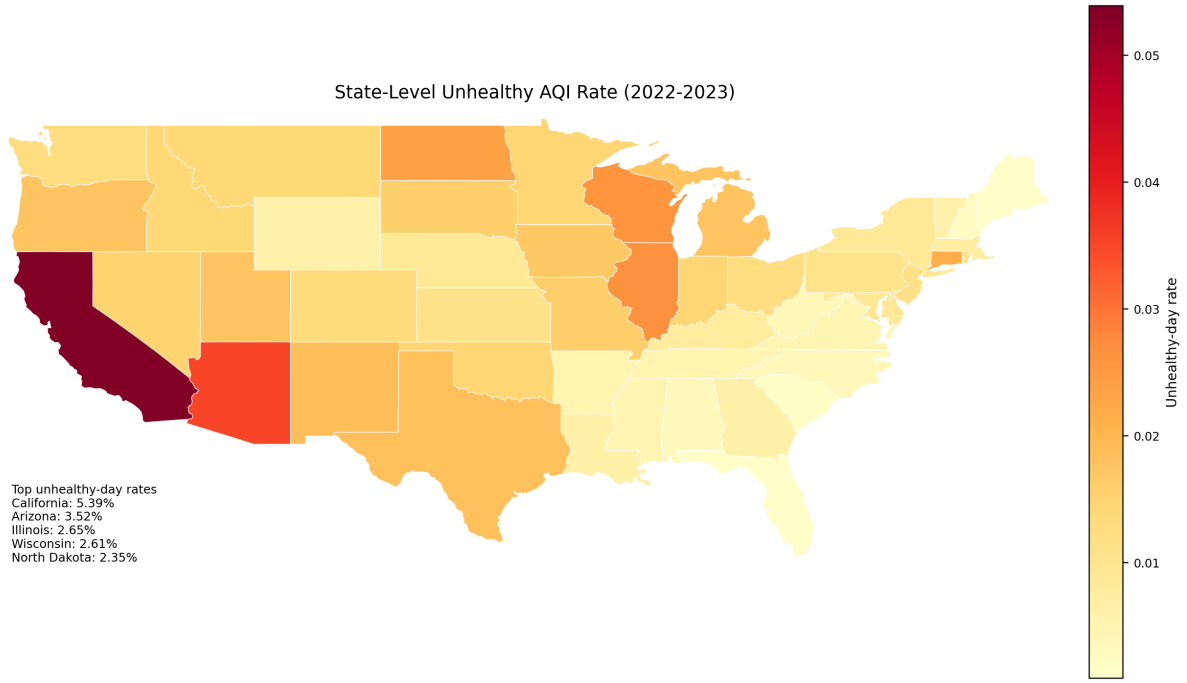


Figure 6: State-level unhealthy-day rate, defined as the proportion of county-days with AQI$>$ 100, computed from the EPA Daily AQI by County data for 2022–2023. The map is generated from Athena-based PySpark summaries.

B Appendix: Source Code

The following listings are imported directly from the Python source files in `../src/`. Consequently, if the corresponding source file is modified, the appendix updates automatically when the report is recompiled.

B.1 Question 1 Source Code

```
"""
MATH70099 - Coursework 2 - Question 1
CID: 06060027
"""

import matplotlib
matplotlib.use('Agg')

import os
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from pyspark.sql.types import DoubleType, LongType

# -- Paths ---------------------------------------------------------------------
HDFS_SANTANDER = "/home/shared_data/santander_trips.csv"
OUTPUT_DIR = os.path.expanduser("~/cw2/outputs/q1")
os.makedirs(OUTPUT_DIR, exist_ok=True)

SEED = 6060027

def create_spark() -> SparkSession:
    """Create and return a SparkSession."""
    spark = (
        SparkSession.builder
        .appName("CW2_Q1_6060027")
        .master("local[*]")
        .getOrCreate()
    )
    spark.sparkContext.setLogLevel("WARN")
    return spark

# -- Part (a) ------------------------------------------------------------------

def load_and_add_time(spark: SparkSession):
    """
    Load santander_trips.csv from HDFS.
    Add:
      time_elapsed = (start_time - min(start_time)) / 60  [minutes]
      time         = time_elapsed + Uniform(0,1) noise   [seed = SEED]

    Returns the resulting DataFrame.
    """

    print("Q1(a): Load the dataset and add the cols requested")

    bikes = (
        spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv(HDFS_SANTANDER)
    )

    print(f"Schema:")
```

```
    bikes.printSchema()
    print(f"Row count: {bikes.count():,}")

    # Compute global minimum of start_time via a cross-join-free approach
    min_start = bikes.agg(F.min("start_time")).collect()[0][0]
    print(f"min(start_time) = {min_start}")

    bikes = bikes.withColumn(
        "time_elapsed",
        (F.col("start_time").cast(DoubleType()) - float(min_start)) / 60.0
    )

    # Reproducible uniform noise: rand(seed=SEED) produces U[0,1)
    bikes = bikes.withColumn(
        "time",
        F.col("time_elapsed") + F.rand(seed=SEED)
    )

    print("\nSample (first 5 rows):")
    bikes.select("start_id", "end_id", "start_time", "time_elapsed",
        "time").show(5)

    return bikes, min_start


# -- Part (b) ------------------------------------------------------------------

def compute_lambda_hat(bikes, min_start):
    """
    Add lambda_hat_i = N_i(T) / T per source station i using a Window
    partitioned by start_id.
    T = max(time_elapsed) over the entire dataset.

    Returns bikes with new column lambda_hat.
    """
    print("Q1(b): Compute lambda_hat per station")

    # Global T (in minutes)
    T = bikes.agg(F.max("time_elapsed")).collect()[0][0]
    print(f"T = max(time_elapsed) = {T:.4f} minutes")

    # N_i(T) = count of trips originating at station i
    station_window = Window.partitionBy("start_id")

    bikes = bikes.withColumn(
        "N_i",
        F.count("*").over(station_window).cast(DoubleType())
    ).withColumn(
        "lambda_hat",
        F.col("N_i") / T
    )

    # Summary table: one row per station (let's cache this)
    station_df = (
        bikes.select("start_id", "lambda_hat")
        .dropDuplicates(["start_id"])
        .cache()
    )

    print("\n5 stations with LARGEST lambda_hat:")
    station_df.orderBy(F.desc("lambda_hat")).show(5)

    print("5 stations with SMALLEST lambda_hat:")
    station_df.orderBy(F.asc("lambda_hat")).show(5)

    return bikes, T, station_df
```

```
124
125
126 # -- Part (c) ------------------------------------------------------------------
127
128 def compute_pval(bikes):
129     """
130     Within each station (partitioned by start_id, ordered by time):
131       inter_arrival = time - lag(time, 1, 0)
132       pval          = exp(-lambda_hat * inter_arrival)
133
134     Report 5 stations with largest and smallest mean pval.
135     Returns bikes with new columns inter_arrival, pval.
136     """
137     print("Q1(c): Compute inter-arrival times and p-values")
138
139     time_window = (
140         Window
141         .partitionBy("start_id")
142         .orderBy("time")
143     )
144
145     bikes = bikes.withColumn(
146         "prev_time",
147         F.lag("time", 1, 0).over(time_window)
148     ).withColumn(
149         "inter_arrival",
150         F.col("time") - F.col("prev_time")
151     ).withColumn(
152         "pval",
153         F.exp(-F.col("lambda_hat") * F.col("inter_arrival"))
154     )
155
156     # Average pval per station
157     avg_pval_df = (
158         bikes.groupBy("start_id")
159         .agg(F.mean("pval").alias("avg_pval"))
160         .cache()
161     )
162
163     print("5 stations with LARGEST average pval:")
164     avg_pval_df.orderBy(F.desc("avg_pval")).show(5)
165
166     print("5 stations with SMALLEST average pval:")
167     avg_pval_df.orderBy(F.asc("avg_pval")).show(5)
168
169     return bikes, avg_pval_df
170
171
172 # -- Part (d) ------------------------------------------------------------------
173
174 def compute_ks_and_boxplot(bikes, avg_pval_df):
175     """
176     For each station, compute the empirical quantile of each pval via
177     percent_rank() over (partition=start_id, order=pval).
178
179     KS statistic per station:
180       D_i = max(|pval - empirical_quantiles|)
181
182     Save a boxplot of D_i values across all stations.
183     """
184     print("Q1(d): Compute KS statistics and boxplot")
185
186     pval_rank_window = (
187         Window
188         .partitionBy("start_id")
189         .orderBy("pval")
```

```
    )

    bikes = bikes.withColumn(
        "empirical_quantiles",
        F.percent_rank().over(pval_rank_window)
    ).withColumn(
        "abs_diff",
        F.abs(F.col("pval") - F.col("empirical_quantiles"))
    )

    # KS stat per station
    ks_df = (
        bikes.groupBy("start_id")
        .agg(F.max("abs_diff").alias("ks_stat"))
        .cache()
    )

    print("KS statistic summary:")
    ks_df.describe("ks_stat").show()

    # Collect KS stats for plotting (one value per station)
    ks_values = [row["ks_stat"] for row in ks_df.collect()]
    ks_arr = np.array(ks_values, dtype=float)

    # Boxplot
    fig, ax = plt.subplots(figsize=(6, 5))
    bp = ax.boxplot(
        ks_arr,
        vert=True,
        patch_artist=True,
        boxprops=dict(facecolor="#4C72B0", color="#2d3e6d"),
        medianprops=dict(color="white", linewidth=2),
        whiskerprops=dict(color="#2d3e6d"),
        capprops=dict(color="#2d3e6d"),
        flierprops=dict(marker="o", markersize=3, linestyle="none",
                        markerfacecolor="#C44E52", alpha=0.4),
    )
    ax.set_title(
        r"KS Statistic $D_i$ per Station" "\n"
        r"(Kolmogorov-Smirnov test of Exp(lambda_hat_i) fit)",
        fontsize=11
    )

    ax.set_ylabel("$D_i = \\max|F_{\\rm emp}(p) - p|$", fontsize=10)
    ax.set_xticks([])
    ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    # Annotate median
    median_val = np.median(ks_arr)
    ax.text(
        1.15, median_val,
        f"median = {median_val:.3f}",
        va="center", fontsize=8, color="#2d3e6d",
        transform=ax.get_yaxis_transform()
    )

    fig.tight_layout()
    out_path = os.path.join(OUTPUT_DIR, "q1d_ks_boxplot.png")
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"Boxplot saved to: {out_path}")

    return ks_df


```

```
# -- Sanity checks ------------------------------------------------------------

def sanity_check_interarrivals(bikes):
    """
    Sanity check: for the station with the LARGEST lambda_hat and the station
    with the SMALLEST lambda_hat, compare the empirical CDF of inter-arrivals
    against the theoretical Exp(lambda_hat) CDF.

    Also prints N_i, lambda_hat, mean inter-arrival, and theoretical mean
        (1/lambda_hat)
    to verify they are consistent.
    """
    print("Q1 sanity check: inter-arrival CDF vs Exp(lambda_hat) fit")

    # Collect data for the two extreme stations
    extremes = (
        bikes.select("start_id", "lambda_hat")
        .dropDuplicates(["start_id"])
    )
    largest_station  = extremes.orderBy(F.desc("lambda_hat")).first()["start_id"]
    smallest_station = extremes.orderBy(F.asc("lambda_hat")).first()["start_id"]

    for station_id, label in [(largest_station, "LARGEST lambda_hat"),
                              (smallest_station, "SMALLEST lambda_hat")]:
        rows = (
            bikes
            .filter(F.col("start_id") == station_id)
            .select("inter_arrival", "lambda_hat")
            .toPandas()
        )
        lam = float(rows["lambda_hat"].iloc[0])
        ia  = rows["inter_arrival"].values

        n_i = len(ia)
        mean_ia   = ia.mean()
        theory_mean = 1.0 / lam if lam > 0 else float("inf")

        print(f"\nStation {station_id} ({label}):")
        print(f"  N_i              = {n_i}")
        print(f"  lambda_hat       = {lam:.6f} trips/min")
        print(f"  Mean inter-arr   = {mean_ia:.4f} min  "
              f"(theoretical 1/lambda = {theory_mean:.4f} min, "
              f"ratio = {mean_ia / theory_mean:.3f})")

        # Empirical CDF vs Exp CDF
        ia_sorted = np.sort(ia)
        ecdf = np.arange(1, n_i + 1) / n_i
        x_grid = np.linspace(0, np.percentile(ia_sorted, 99), 300)
        exp_cdf = 1 - np.exp(-lam * x_grid)

        ks_stat = np.max(np.abs(
            (np.arange(1, n_i + 1) / n_i) -
            (1 - np.exp(-lam * ia_sorted))
        ))
        print(f"  KS statistic     = {ks_stat:.4f}")

        fig, ax = plt.subplots(figsize=(7, 4))
        ax.step(ia_sorted, ecdf, where="post", color="#4C72B0", lw=1.5,
                label=f"Empirical CDF (n={n_i})")
        ax.plot(x_grid, exp_cdf, color="#C44E52", lw=2, linestyle="--",
                label=f"Exp(lambda_hat={lam:.4f}) CDF")
        ax.set_xlabel("Inter-arrival time (minutes)", fontsize=10)
        ax.set_ylabel("CDF", fontsize=10)
        ax.set_title(
            f"Inter-arrival CDF vs Exp fit - Station {station_id} ({label})\n"
            f"KS={ks_stat:.4f},  "
```

```
            f"lambda_hat={lam:.5f},  "
            f"mean_obs={mean_ia:.2f} vs 1/lambda={theory_mean:.2f}",
            fontsize=9
        )
        ax.legend(fontsize=9)
        ax.grid(linestyle="--", alpha=0.4)
        fig.tight_layout()
        out_path = os.path.join(OUTPUT_DIR,
                                f"q1_sanity_station{station_id}.png")
        fig.savefig(out_path, dpi=150)
        plt.close(fig)
        print(f"  Plot saved: {out_path}")

    print("\nSanity check complete.")


# -- Main ------------------------------------------------------------------

def main():
    spark = create_spark()

    # (a)
    bikes, min_start = load_and_add_time(spark)

    # (b)
    bikes, T, station_df = compute_lambda_hat(bikes, min_start)

    # (c)
    bikes, avg_pval_df = compute_pval(bikes)

    # (d)
    ks_df = compute_ks_and_boxplot(bikes, avg_pval_df)

    # Sanity checks: inter-arrival CDF vs Exp fit for 2 extreme stations
    sanity_check_interarrivals(bikes)

    print("\nAll Q1 tasks complete.")
    spark.stop()


if __name__ == "__main__":
    main()
```

Listing 9: Full source code for Question 1 (`src/q1_window_functions.py`)

B.2 Question 2 Source Code

```
"""
MATH70099 - Coursework 2 - Question 2
CID: 06060027

Bayesian inference on the precision matrix of a zero-mean multivariate
Gaussian model using the Wishart conjugate prior. Theory, derivations,
and discussion are in the accompanying coursework report.
"""

import matplotlib
matplotlib.use('Agg')

import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import linalg
from scipy.stats import wishart, chi2
```

```python

# -- Paths ------------------------------------------------------------------
PROJECT_DIR = Path(__file__).resolve().parent.parent
LOCAL_DATA_DIR = PROJECT_DIR.parent / "data"
SHARED_DATA_DIR = Path(os.environ.get("CW2_SHARED_DATA_DIR",
    "/home/shared_data/2026/cw2"))
DATA_DIR = SHARED_DATA_DIR if SHARED_DATA_DIR.exists() else LOCAL_DATA_DIR
OUTPUT_DIR = Path(os.environ.get("Q2_OUTPUT_DIR", PROJECT_DIR / "outputs_athena"
    / "q2"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

GENE_CSV = DATA_DIR / "gene_expression.csv"

# -- Constants --------------------------------------------------------------
SEED    = 6060027
rng     = np.random.default_rng(SEED)
D       = 6           # dimensionality
NU      = 6           # prior df (= D, so the prior is proper for nu > d-1)
N_FULL  = 500         # total observations
S_DRAW  = 1000        # Monte Carlo samples
B       = 5           # number of batches
# batch sizes as given in the problem
N_BATCH = np.array([53, 110, 149, 103, 85], dtype=float)  # sum = 500


# -- Data loading -----------------------------------------------------------

def load_gene_data(path: str):
    """
    Load gene_expression.csv.
    Returns:
      X       : (500, 6) float array of features
      batches: (500,)   int array of batch labels in {0,1,2,3,4}
    """
    print("Q2: Load gene expression data")

    import csv
    rows, labels = [], []
    with open(path, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            rows.append([float(row[f"x{j}"]) for j in range(1, D + 1)])
            labels.append(int(row["batch"]))

    X = np.array(rows, dtype=float)
    batches = np.array(labels, dtype=int)
    print(f"Loaded X: {X.shape},  batches unique: {np.unique(batches)}")
    print(f"Batch sizes: {[int((batches == b).sum()) for b in range(B)]}")
    return X, batches


# -- Posterior parameters ---------------------------------------------------

def full_posterior_params(X: np.ndarray):
    """
    Compute full posterior parameters.
    Lambda|X ~ W_d(nu+n, (I + S)^(-1))

    Returns: (df_full, scale_full)
    """
    n = X.shape[0]
    S = X.T @ X                           # (d,d) scatter
    df_full = NU + n                      # 506
    scale_full = linalg.inv(np.eye(D) + S)
    return df_full, scale_full, S

```

```
84
85  def sub_posterior_params(X: np.ndarray, batches: np.ndarray):
86      """
87      Compute sub-posterior parameters for each batch b.
88
89      Sub-posterior: Lambda | X_b  ~  W_d(df_b, scale_b)
90        df_b    = 7 - n_b/n + n_b
91        scale_b = ((n_b/n)*I + S_b)^{-1}
92
93      Returns: list of (df_b, scale_b) for b in 0..4
94      """
95      n = float(X.shape[0])
96      params = []
97      for b in range(B):
98          mask = batches == b
99          X_b  = X[mask]
100         n_b  = float(X_b.shape[0])
101         S_b  = X_b.T @ X_b
102         df_b = 7.0 - n_b / n + n_b
103         scale_b = linalg.inv((n_b / n) * np.eye(D) + S_b)
104         params.append((df_b, scale_b))
105         print(f"  Batch {b}: n_b={int(n_b)}, df_b={df_b:.4f}, "
106               f"scale_b trace={np.trace(scale_b):.6f}")
107     return params
108
109
110 # -- Part (a): CMC ------------------------------------------------------------
111
112 def sample_sub_posteriors(sub_params, s: int = S_DRAW):
113     """
114     Draw S samples from each sub-posterior.
115     Returns list of arrays of shape (S, D, D).
116     """
117     samples = []
118     for b, (df_b, scale_b) in enumerate(sub_params):
119         samp = wishart.rvs(df=df_b, scale=scale_b, size=s,
120                            random_state=rng.integers(1e9))
121         # wishart.rvs returns (S, D, D) when size=S and df > D-1
122         samples.append(samp)
123         print(f"  Sub-posterior {b}: sampled {samp.shape[0]} matrices")
124     return samples
125
126
127 def cmc_scheme_i(sub_samples, batches: np.ndarray):
128     """
129     Weighting scheme (i): precision-weighted average.
130     Lambda_tilde^(s) = sum_b (n_b/n) * Lambda^(b,s)
131
132     Returns: array (S, D, D)
133     """
134     # Use actual batch sizes from data (not hardcoded)
135     n_batch_actual = np.array([(batches == b).sum() for b in range(B)],
        dtype=float)
136     n = n_batch_actual.sum()
137     combined = np.zeros_like(sub_samples[0])
138     for b, samp in enumerate(sub_samples):
139         w_b = n_batch_actual[b] / n
140         combined += w_b * samp
141     return combined
142
143
144 def cmc_scheme_ii(sub_samples):
145     """
146     Weighting scheme (ii): variance-weighted average (element-wise).
147     Omega_b_jk = Var[Lambda^(b,s)_jk]  (variance over S samples)
```

```
    Lambda_hat^(s)_jk = [sum_b Lambda^(b,s)_jk / Omega_b_jk] / [sum_b 1 /
        Omega_b_jk]

    Returns: array (S, D, D)
    """
    S  = sub_samples[0].shape[0]
    # Compute element-wise variance for each batch
    omegas = []   # list of (D, D) variance matrices
    for samp in sub_samples:
        omega = np.var(samp, axis=0, ddof=1)  # (D, D)
        # Avoid division by zero
        omega = np.where(omega < 1e-15, 1e-15, omega)
        omegas.append(omega)

    # Weighted sum numerator and denominator
    numerator   = np.zeros((S, D, D))
    denominator = np.zeros((D, D))

    for b, (samp, omega) in enumerate(zip(sub_samples, omegas)):
        inv_omega    = 1.0 / omega          # (D, D)
        numerator   += samp * inv_omega     # broadcast (S,D,D) * (D,D)
        denominator += inv_omega

    combined = numerator / denominator      # (S, D, D)
    return combined


def upper_tri_samples(samples_3d: np.ndarray):
    """
    Extract the upper triangular elements (including diagonal) from each
    (D,D) matrix in a (S,D,D) array.
    Returns: (S, D*(D+1)//2) array with column names.
    """
    idx_r, idx_c = np.triu_indices(D)
    col_names = [f"/Lambda_{r+1}{c+1}" for r, c in zip(idx_r, idx_c)]
    flat = samples_3d[:, idx_r, idx_c]  # (S, 21)
    return flat, col_names


def pairplot_precision(scheme_i_samples, scheme_ii_samples, full_samples,
                       output_dir: str):
    """
    Pairplot of the D*(D+1)/2 = 21 upper-triangular elements for:
      - CMC scheme (i)
      - CMC scheme (ii)
      - Full posterior
    Saves one PNG per scheme for clarity (21x21 pairplot is very large).
    Instead, we plot diagonal elements only in one figure and off-diagonal
    pair (1,2) as an example scatter in another.
    """
    flat_i,    cols = upper_tri_samples(scheme_i_samples)
    flat_ii,   _    = upper_tri_samples(scheme_ii_samples)
    flat_full, _    = upper_tri_samples(full_samples)

    # -- Figure 1: marginal densities of all 21 elements ----------------------
    n_elem = len(cols)
    fig, axes = plt.subplots(3, n_elem, figsize=(n_elem * 2.2, 7), sharey=False)

    colours = {"CMC-I": "#4C72B0", "CMC-II": "#DD8452", "Full": "#55A868"}
    datasets = [
        ("CMC-I",  flat_i),
        ("CMC-II", flat_ii),
        ("Full",   flat_full),
    ]

    for row_idx, (label, flat) in enumerate(datasets):
```

```
        for col_idx, cname in enumerate(cols):
            ax = axes[row_idx, col_idx]
            ax.hist(flat[:, col_idx], bins=40, color=colours[label],
                    alpha=0.7, density=True, edgecolor="none")
            if row_idx == 0:
                ax.set_title(cname, fontsize=6, rotation=45, ha="right")
            if col_idx == 0:
                ax.set_ylabel(label, fontsize=8)
            ax.tick_params(labelsize=5)

    fig.suptitle("Marginal distributions of /Lambda upper-triangular elements\n"
                 "(CMC scheme I, CMC scheme II, Full posterior)", fontsize=10)
    fig.tight_layout()
    out_path = os.path.join(output_dir, "q2a_marginals.png")
    fig.savefig(out_path, dpi=120)
    plt.close(fig)
    print(f"Marginals plot saved: {out_path}")

    # -- Figure 2: 3-way scatter for diagonal elements (/Lambda_11 .. /Lambda_66)
        ---------
    diag_idx = [cols.index(f"/Lambda_{j}{j}") for j in range(1, D + 1)]
    fig, axes = plt.subplots(2, 3, figsize=(14, 10), sharey=False)
    axes_flat = axes.flatten()
    for k, di in enumerate(diag_idx):
        ax = axes_flat[k]
        for label, flat, c in [
            ("CMC-I",  flat_i,    "#4C72B0"),
            ("CMC-II", flat_ii,   "#DD8452"),
            ("Full",   flat_full, "#55A868"),
        ]:
            ax.hist(flat[:, di], bins=40, color=c, alpha=0.5,
                    density=True, label=label, edgecolor="none")
        ax.set_title(cols[di], fontsize=13)
        ax.set_xlabel("Value", fontsize=11)
        ax.set_ylabel("Density", fontsize=11)
        ax.tick_params(labelsize=10)
        if k == 0:
            ax.legend(fontsize=10)
    fig.suptitle("Diagonal elements of $\\Lambda$: CMC vs Full posterior",
        fontsize=14)
    fig.tight_layout()
    out_path = os.path.join(output_dir, "q2a_diagonal_compare.png")
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"Diagonal comparison saved: {out_path}")

    # -- Figure 3: 2D scatter /Lambda_11 vs /Lambda_12
        ------------------------------------
    fig, axes = plt.subplots(1, 3, figsize=(12, 4), sharex=False, sharey=False)
    for ax, (label, flat, c) in zip(axes, [
        ("CMC-I",  flat_i,    "#4C72B0"),
        ("CMC-II", flat_ii,   "#DD8452"),
        ("Full",   flat_full, "#55A868"),
    ]):
        ax.scatter(flat[:, cols.index("/Lambda_11")],
                   flat[:, cols.index("/Lambda_12")],
                   s=3, alpha=0.3, color=c)
        ax.set_xlabel("/Lambda_11", fontsize=9)
        ax.set_ylabel("/Lambda_12", fontsize=9)
        ax.set_title(label, fontsize=10)
    fig.suptitle("Joint distribution of (/Lambda_11, /Lambda_12): CMC vs Full",
        fontsize=10)
    fig.tight_layout()
    out_path = os.path.join(output_dir, "q2a_scatter_L11_L12.png")
    fig.savefig(out_path, dpi=120)
    plt.close(fig)
```

```
    print(f"Scatter plot saved: {out_path}")


def compare_moments(scheme_i_samples, scheme_ii_samples, full_samples,
                    cols):
    """Print mean and variance comparison across schemes."""
    print("\n-- Moment Comparison (upper-triangular elements) --")
    flat_i,    _ = upper_tri_samples(scheme_i_samples)
    flat_ii,   _ = upper_tri_samples(scheme_ii_samples)
    flat_full, _ = upper_tri_samples(full_samples)

    print(f"{'Element':<10} {'Mean (I)':>12} {'Mean (II)':>12} {'Mean
        (Full)':>12} "
          f"{'Var (I)':>12} {'Var (II)':>12} {'Var (Full)':>12}")
    print("-" * 84)
    for j, col in enumerate(cols):
        print(f"{col:<10} "
              f"{flat_i[:,j].mean():>12.5f} "
              f"{flat_ii[:,j].mean():>12.5f} "
              f"{flat_full[:,j].mean():>12.5f}  "
              f"{flat_i[:,j].var():>11.5f} "
              f"{flat_ii[:,j].var():>12.5f} "
              f"{flat_full[:,j].var():>12.5f}")


def run_part_a(X, batches):
    """Execute Q2(a): CMC algorithm."""
    print("Q2(a): CMC Algorithm")

    # Sub-posterior parameters
    print("\nSub-posterior parameters:")
    sub_params = sub_posterior_params(X, batches)

    # Full posterior parameters
    df_full, scale_full, S = full_posterior_params(X)
    print(f"\nFull posterior: df={df_full}, scale
        trace={np.trace(scale_full):.6f}")

    # Sample
    print("\nSampling sub-posteriors...")
    sub_samples = sample_sub_posteriors(sub_params, s=S_DRAW)

    print("Sampling full posterior...")
    full_samples = wishart.rvs(df=df_full, scale=scale_full, size=S_DRAW,
                               random_state=rng.integers(1e9))
    if full_samples.ndim == 2:
        full_samples = full_samples[np.newaxis]

    # CMC combinations
    print("\nComputing CMC scheme (i)  - precision-weighted average...")
    si_samples = cmc_scheme_i(sub_samples, batches)

    print("Computing CMC scheme (ii) - variance-weighted average...")
    sii_samples = cmc_scheme_ii(sub_samples)

    # Visualise
    print("\nGenerating plots...")
    pairplot_precision(si_samples, sii_samples, full_samples, OUTPUT_DIR)

    # Moment comparison
    _, cols = upper_tri_samples(full_samples)
    compare_moments(si_samples, sii_samples, full_samples, cols)

    return si_samples, sii_samples, full_samples


```

```python
# -- Part (b): Subsampling estimators ----------------------------------------

def run_part_b():
    """Q2(b): Subsampling estimators; derivation and MSE analysis are in the report."""
    print("Q2(b): Subsampling Estimators (see the report for the full derivation)")


# -- Part (c): CMC appropriateness -------------------------------------------

def run_part_c():
    """Q2(c): Inflated batch-specific prior and CMC; discussion is in the report."""
    print("Q2(c): Inflated sub-posterior / CMC discussion (see the report)")


# -- Part (d): Distributed gradient descent with doubly stochastic W -----------

def run_part_d():
    """
    Q2(d): Doubly stochastic weight matrix and distributed gradient descent.
    """
    print("\n" + "=" * 70)
    print("Q2(d): Doubly Stochastic Weight Matrix and Distributed GD")
    print("=" * 70)

    # Weight matrix as specified in the question
    W = np.array([
        [1/4, 1/4, 1/4, 1/4, 0  ],
        [1/4, 1/4, 1/4, 1/4, 0  ],
        [0,   0,   1/2, 1/4, 1/4],
        [1/4, 1/4, 0,   1/4, 1/4],
        [1/4, 1/4, 0,   0,   1/2],
    ])

    print("\nWeight matrix W:")
    print(np.array2string(W, precision=4, suppress_small=True))

    # Verify doubly stochastic
    row_sums = W.sum(axis=1)
    col_sums = W.sum(axis=0)
    print(f"\nRow sums: {row_sums}")
    print(f"Col sums: {col_sums}")

    is_row_stochastic = np.allclose(row_sums, 1.0)
    is_col_stochastic = np.allclose(col_sums, 1.0)
    is_nonneg         = np.all(W >= 0)

    print(f"\nAll rows sum to 1: {is_row_stochastic}")
    print(f"All cols sum to 1: {is_col_stochastic}")
    print(f"All entries  >=  0:   {is_nonneg}")
    print(f"W is doubly stochastic: {is_row_stochastic and is_col_stochastic and is_nonneg}")

    # Distributed GD update and convergence; derivation is given in the report

    # Compute spectral gap of W
    # Use eigvals (not eigvalsh) since W is not symmetric
    eigenvalues = np.linalg.eigvals(W)
    eigenvalues_sorted = np.sort(np.abs(eigenvalues))[::-1]
    print(f"Eigenvalues of W (magnitude, descending): {eigenvalues_sorted}")
    if len(eigenvalues_sorted) > 1:
        spectral_gap = 1.0 - eigenvalues_sorted[1]
        print(f"Spectral gap (1 - |/Lambda_2|) = {spectral_gap:.4f}")

```

```python

# -- Main ----------------------------------------------------------------------

def main():
    print("=" * 70)
    print("MATH70099 CW2 - Q2: Bayesian Wishart Inference")
    print("=" * 70)

    # Load data
    X, batches = load_gene_data(GENE_CSV)

    # (a) CMC
    si_samples, sii_samples, full_samples = run_part_a(X, batches)

    # (b) Subsampling estimators (analytical, no extra computation needed)
    run_part_b()

    # (c) Appropriateness of CMC
    run_part_c()

    # (d) Doubly stochastic W and distributed GD
    run_part_d()

    print("\nAll Q2 tasks complete.")


if __name__ == "__main__":
    main()
```

Listing 10: Full source code for Question 2 (`src/q2_bayes_precision.py`)

B.3 Question 3 Source Code

```python
"""
MATH70099 - Coursework 2 - Question 3
CID: 06060027

Markov Mixture Model, Variational Inference (CAVI), and
EWMA Changepoint Detection on drone trajectory data.
"""

import matplotlib
matplotlib.use('Agg')

import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import digamma, gammaln
from scipy.stats import chi2

# -- Paths ---------------------------------------------------------------------
PROJECT_DIR = Path(__file__).resolve().parent.parent
LOCAL_DATA_DIR = PROJECT_DIR.parent / "data"
SHARED_DATA_DIR = Path(os.environ.get("CW2_SHARED_DATA_DIR",
    "/home/shared_data/2026/cw2"))
DATA_DIR = SHARED_DATA_DIR if SHARED_DATA_DIR.exists() else LOCAL_DATA_DIR
OUTPUT_DIR = Path(os.environ.get("Q3_OUTPUT_DIR", PROJECT_DIR / "outputs_athena"
    / "q3"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

DRONE_NPY = DATA_DIR / "drone_trajectories.npy"

# -- Constants -----------------------------------------------------------------
SEED = 6060027
rng  = np.random.default_rng(SEED)
```

```python

M_GRID = 16  # 4x4 grid, states labelled 1..16
# Initial positions that appear in the data: {1, 2, 5, 12, 15, 16}
OBS_STATES = [1, 2, 5, 12, 15, 16]

# mu_null: null distribution over all 16 positions (1-indexed)
# Positive mass appears only at positions 1, 2, 5, 12, 15, and 16.
MU_NULL = np.array([
    1/4, 1/8, 0, 0, 1/8, 0, 0, 0,
    0,   0,   0, 1/8, 0, 0, 1/8, 1/4
])  # length 16, indices 0..15 correspond to positions 1..16
assert len(MU_NULL) == M_GRID
assert np.isclose(MU_NULL.sum(), 1.0), f"mu_null sums to {MU_NULL.sum()}"


# -- Part (a): Sufficient Statistics ------------------------------------------

def print_sufficient_statistics():
    """Sufficient statistics for the mixture model; derivation is in the
       report."""
    print(
        "Q3(a): the code-level sufficient statistics are the initial-state counts
            "
        "T0 and the labelled transition-count tensors T_{k,m,m'} derived in the
           report."
    )


# -- Part (b): CAVI Variational Inference -------------------------------------

def print_cavi_updates():
    """CAVI update equations; derivation is in the report."""
    print(
        "Q3(b): the report derives the closed-form CAVI updates; the
           implementation "
        "below realises those updates with transition counts, digamma
           expectations, "
        "and ELBO monitoring."
    )


def compute_transition_counts(trajectory: np.ndarray, M: int) -> np.ndarray:
    """
    Compute transition count matrix N[m, m'] for a single trajectory.
    trajectory: 1D array of states (0-indexed, 0..M-1)
    Returns: (M, M) int array
    """
    N = np.zeros((M, M), dtype=float)
    for t in range(1, len(trajectory)):
        N[trajectory[t - 1], trajectory[t]] += 1
    return N


def cavi_elbo(pi_ik, rho_0, rho_km,
              X_onehot, trans_counts, alpha: float, M: int, K: int):
    """
    Compute the ELBO for the Markov mixture model.

    Prior: z_i ~ Cat(1/K,...,1/K)  [fixed uniform, no latent pi]
           phi_0   ~ Dir(alpha*1_M)
           phi_k,m ~ Dir(alpha*1_M)

    Parameters
    ----------
    pi_ik       : (n, K) responsibility matrix
    rho_0       : (M,) Dirichlet params for phi_0
```

```
    rho_km       : (K, M, M) Dirichlet params for phi_{k,m}
    X_onehot     : (n,) initial states (0-indexed)
    trans_counts: (n, M, M) per-trajectory transition counts
    alpha        : symmetric Dirichlet prior concentration
    M, K         : dimensions

    Returns
    -------
    float : ELBO value
    """
    n = pi_ik.shape[0]
    eps = 1e-300

    # E[log p(z_i)] = sum_i sum_k pi_{ik} * log(1/K) for the fixed uniform prior
    elbo = np.sum(pi_ik) * (-np.log(K))  # = n * (-log K)

    # E[log p(x_{i,1} | phi_0)]
    E_log_phi0 = digamma(rho_0) - digamma(rho_0.sum())
    elbo += np.sum(E_log_phi0[X_onehot])

    # E[log p(x_{i,t} | x_{i,t-1}, z_i, phi)]
    # = sum_i sum_k pi_{ik} sum_{m,m'} N_{i,m,m'} * E[log phi_{k,m,m'}]
    for k in range(K):
        for m in range(M):
            if rho_km[k, m].sum() > 0:
                E_log_phi_km = (digamma(rho_km[k, m])
                                - digamma(rho_km[k, m].sum()))
                elbo += np.einsum(
                    'i,im,m->',
                    pi_ik[:, k],
                    trans_counts[:, m, :],
                    E_log_phi_km
                )

    # -KL[q(phi_0) || p(phi_0)]
    alpha0_prior = np.full(M, alpha)
    elbo += _dirichlet_kl_neg(rho_0, alpha0_prior)

    # sum_{k,m} -KL[q(phi_{k,m}) || p(phi_{k,m})]
    for k in range(K):
        for m in range(M):
            alpha_km_prior = np.full(M, alpha)
            elbo += _dirichlet_kl_neg(rho_km[k, m], alpha_km_prior)

    # Entropy of q(z_i): + sum_i H[q(z_i)]
    elbo += -np.sum(pi_ik * np.log(pi_ik + eps))

    return elbo


def _dirichlet_kl_neg(q_alpha: np.ndarray, p_alpha: np.ndarray) -> float:
    """
    -KL[Dir(q_alpha) || Dir(p_alpha)]
    = log B(q_alpha) - log B(p_alpha)
      + sum_m (p_alpha_m - q_alpha_m) * (psi(q_alpha_m) - psi(sum q_alpha_m))
    """
    def log_beta(a):
        return gammaln(a).sum() - gammaln(a.sum())

    psi_q = digamma(q_alpha) - digamma(q_alpha.sum())
    return (log_beta(q_alpha) - log_beta(p_alpha)
            + np.dot(p_alpha - q_alpha, psi_q))


def run_cavi(X_init: np.ndarray, trans_counts_all: np.ndarray,
             K: int, M: int, alpha: float = 1.0,
```

```
            max_iter: int = 200, tol: float = 1e-6,
            verbose: bool = True):
    """
    Run CAVI for the Markov mixture model.

    Prior: z_i ~ Cat(1/K,...,1/K) [fixed, no latent pi variable]
           phi_0, phi_{k,m} ~ Dir(alpha*1_M)

    Parameters
    ----------
    X_init          : (n,) int array, initial state per trajectory (0-indexed)
    trans_counts_all: (n, M, M) transition count matrices
    K               : number of mixture components
    M               : number of states
    alpha           : symmetric Dirichlet concentration
    max_iter        : maximum CAVI iterations
    tol             : convergence threshold on ELBO change

    Returns
    -------
    pi_ik  : (n, K) final responsibilities
    rho_0  : (M,) final Dirichlet params for phi_0
    rho_km : (K, M, M) final Dirichlet params for phi_{k,m}
    elbos  : list of ELBO values per iteration
    """
    n = X_init.shape[0]

    # Initialise responsibilities uniformly with Dirichlet noise
    pi_ik = rng.dirichlet(np.ones(K), size=n)  # (n, K)

    # Compute T_{0,m}: initial-state counts (fixed, updated once)
    T0 = np.zeros(M)
    for s in X_init:
        T0[s] += 1
    rho_0 = alpha + T0  # (M,); does not depend on z, so fixed

    # Initialise rho_km
    rho_km = np.full((K, M, M), alpha)

    elbos = []

    for it in range(max_iter):
        # Update q*(phi_{k,m})
        # rho_{k,m,m'} = alpha + sum_i pi_{ik} * N_{i,m,m'}
        rho_km = np.full((K, M, M), alpha)
        for k in range(K):
            rho_km[k] = alpha + np.einsum('i,imn->mn',
                                          pi_ik[:, k], trans_counts_all)

        # Update q*(z_i)
        # z_i ~ Cat(1/K): prior term log(1/K) is constant across k,
        # so it cancels in the softmax and is omitted.
        E_log_phi0 = digamma(rho_0) - digamma(rho_0.sum())

        # log_resp[i, k] = E[log phi_{0, x_{i,1}}] + sum_t E[log phi_{k,...}]
        log_resp = E_log_phi0[X_init, np.newaxis].repeat(K, axis=1)  # (n, K)

        for k in range(K):
            E_log_phi_k = np.zeros((M, M))
            for m in range(M):
                row_sum = rho_km[k, m].sum()
                if row_sum > 0:
                    E_log_phi_k[m] = (digamma(rho_km[k, m])
                                      - digamma(row_sum))
            log_resp[:, k] += np.einsum('imn,mn->i', trans_counts_all,
                                        E_log_phi_k)
```

```
        # Numerical stability + normalise
        log_resp -= log_resp.max(axis=1, keepdims=True)
        pi_ik = np.exp(log_resp)
        pi_ik /= pi_ik.sum(axis=1, keepdims=True)

        # ELBO
        elbo = cavi_elbo(pi_ik, rho_0, rho_km,
                         X_init, trans_counts_all, alpha, M, K)
        elbos.append(elbo)

        if verbose and (it % 20 == 0 or it < 5):
            print(f"  Iter {it:4d}: ELBO = {elbo:.4f}")

        if it > 0:
            delta = elbos[-1] - elbos[-2]
            # CAVI should give a non-decreasing ELBO; flag clear numerical
                violations
            if delta < -1e-6:
                print(f"  WARNING: ELBO decreased at iter {it} by {delta:.2e} "
                      f"(numerical precision issue, expected near-zero)")
            if abs(delta) < tol:
                if verbose:
                    print(f"  Converged at iteration {it} (|DeltaELBO| =
                        {abs(delta):.2e} < {tol})")
                break

    # Validate monotonicity after convergence
    elbo_arr = np.array(elbos)
    violations = np.sum(np.diff(elbo_arr) < -1e-4)
    if verbose:
        print(f"  ELBO monotonicity check: {violations} violation(s) (expected
            0)")
        print(f"  ELBO range: [{elbo_arr.min():.4f}, {elbo_arr.max():.4f}], "
              f"Delta_final = {elbo_arr[-1] - elbo_arr[0]:.4f}")

    return pi_ik, rho_0, rho_km, elbos


def generate_synthetic_data(n: int = 50, K_true: int = 2, M: int = 4,
                            T_len: int = 10):
    """
    Generate synthetic Markov mixture data for testing CAVI.

    Returns
    -------
    X_init        : (n,) initial states (0-indexed)
    trans_counts  : (n, M, M) transition counts
    true_labels   : (n,) true cluster assignments
    """
    # True parameters
    phi0_true   = rng.dirichlet(np.ones(M))
    phi_km_true = np.zeros((K_true, M, M))
    for k in range(K_true):
        for m in range(M):
            phi_km_true[k, m] = rng.dirichlet(np.ones(M) * (k + 1))
    pi_true = rng.dirichlet(np.ones(K_true))

    X_init = np.zeros(n, dtype=int)
    trans_counts = np.zeros((n, M, M), dtype=float)
    true_labels  = np.zeros(n, dtype=int)

    for i in range(n):
        k = rng.choice(K_true, p=pi_true)
        true_labels[i] = k
        x = rng.choice(M, p=phi0_true)
```

```
        X_init[i] = x
        for t in range(1, T_len):
            x_prev = x
            x = rng.choice(M, p=phi_km_true[k, x_prev])
            trans_counts[i, x_prev, x] += 1

    return X_init, trans_counts, true_labels


def run_part_b_cavi():
    """Run a small synthetic check of the closed-form CAVI updates."""
    print("\n" + "=" * 70)
    print("Q3(b): Supplementary synthetic verification of the CAVI updates")
    print("=" * 70)

    # Generate synthetic data
    K_test, n_test, M_test = 2, 50, 4
    print(f"\nGenerating synthetic data: n={n_test}, K={K_test}, M={M_test}")
    X_init, trans_counts, true_labels = generate_synthetic_data(
        n=n_test, K_true=K_test, M=M_test, T_len=15
    )
    print(f"X_init shape: {X_init.shape}")
    print(f"trans_counts shape: {trans_counts.shape}")
    print(f"True label distribution: {np.bincount(true_labels)}")

    # Run CAVI
    print(f"\nRunning CAVI with K={K_test}...")
    pi_ik, rho_0, rho_km, elbos = run_cavi(
        X_init, trans_counts, K=K_test, M=M_test,
        alpha=1.0, max_iter=300, tol=1e-6, verbose=True
    )

    # Evaluate: cluster assignment
    z_hat = pi_ik.argmax(axis=1)
    print(f"\nEstimated cluster sizes: {np.bincount(z_hat, minlength=K_test)}")
    mixing_weights = pi_ik.mean(axis=0)
    print(f"Estimated mixing weights (empirical): {mixing_weights}")

    # Plot ELBO
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(elbos, color="#4C72B0", lw=2, marker="o", markersize=3)
    ax.set_xlabel("CAVI Iteration", fontsize=11)
    ax.set_ylabel("ELBO", fontsize=11)
    ax.set_title(f"CAVI Convergence (K={K_test}, n={n_test}, M={M_test})",
                 fontsize=12)
    ax.grid(linestyle="--", alpha=0.5)
    fig.tight_layout()
    out_path = OUTPUT_DIR / "q3b_elbo_convergence.png"
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"\nELBO convergence plot saved: {out_path}")

    return pi_ik, elbos


# -- Part (c): EWMA Changepoint Detection ---------------------------------------

def run_ewma_changepoint(data_path: Path):
    """
    EWMA changepoint detection on drone trajectory initial positions.

    EWMA statistic used in part (c):
      Z_t = (1-rho) Z_{t-1} + rho e_t,   Z_0 = mu
      Sigma_Z_t = rho/(2-rho) * [1-(1-rho)^(2t)] * Sigma
      S_t = (Z_t - mu)^T Sigma_Z_t^+ (Z_t - mu)

```

```
    where Sigma = diag(mu) - mu mu^T for one-hot observations,
    and ^+ denotes the Moore-Penrose pseudoinverse.
    """
    print("\n" + "=" * 70)
    print("Q3(c): EWMA Changepoint Detection on Drone Trajectories")
    print("=" * 70)

    # Load data
    print(f"Loading: {data_path}")
    try:
        raw = np.load(data_path)
    except FileNotFoundError:
        print(f"WARNING: {data_path} not found. Using synthetic fallback data.")
        rng_local = np.random.default_rng(SEED)
        p1 = MU_NULL[[0, 1, 4, 11, 14, 15]] / MU_NULL[[0, 1, 4, 11, 14, 15]].sum()
        p2 = np.array([0.05, 0.05, 0.05, 0.55, 0.15, 0.15])
        states_null = [1, 2, 5, 12, 15, 16]
        part1 = rng_local.choice(states_null, size=300, p=p1)
        part2 = rng_local.choice(states_null, size=200, p=p2)
        raw = np.concatenate([part1, part2])

    print(f"Data shape: {raw.shape}, dtype: {raw.dtype}")
    print(f"Unique values: {np.unique(raw)}")

    n = len(raw)

    # One-hot encode states 1..16 to indices 0..15
    E = np.zeros((n, M_GRID), dtype=float)
    for t, state in enumerate(raw):
        E[t, int(state) - 1] = 1.0

    # EWMA parameters
    r = 0.1
    mu = MU_NULL.copy()  # (16,) null mean

    # Multinomial covariance for one-hot observations:
    #   Sigma_{kl} = mu_k delta_{kl} - mu_k mu_l
    Sigma = np.diag(mu) - np.outer(mu, mu)  # (16,16)

    # Pseudoinverse of Sigma (rank = number of non-zero states - 1 = 5)
    Sigma_pinv = np.linalg.pinv(Sigma)  # (16,16)

    # Rank for chi-squared degrees of freedom
    eigvals = np.linalg.eigvalsh(Sigma)
    rank_S = int((eigvals > 1e-10).sum())
    print(f"Rank of Sigma: {rank_S}  (= #observed states - 1 = 5 expected)")

    # Control limit: chi2 at a high percentile with df = rank(Sigma)
    alpha_cl = 0.999
    L = chi2.ppf(alpha_cl, df=rank_S)
    print(f"Control limit L = chi2.ppf({alpha_cl}, df={rank_S}) = {L:.4f}")

    # Run the EWMA control chart
    # S_t = (Z_t - mu)^T [rho/(2-rho)(1-(1-rho)^(2t)) Sigma]^+ (Z_t - mu)
    #     = (Z_t - mu)^T Sigma^+ (Z_t - mu) / rho_factor
    Z = mu.copy()    # Z_0 = mu
    S = np.zeros(n)
    S[0] = 0.0       # Z_0 = mu implies S_0 = 0

    for i in range(1, n):
        rho_factor = r / (2 - r) * (1 - (1 - r) ** (2 * i))
        Z = (1 - r) * Z + r * E[i]
        diff = Z - mu
        S[i] = float(diff @ (Sigma_pinv / rho_factor) @ diff)

    # Detect changepoint as the first exceedance of L
```

```
    exceed_idx = np.where(S > L)[0]
    if len(exceed_idx) > 0:
        cp = int(exceed_idx[0])
        print(f"Changepoint detected at t={cp} (obs {cp+1}), S_t = {S[cp]:.4f}")
    else:
        cp = None
        print("No changepoint detected (S_t never exceeded control limit).")

    # Plot
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(np.arange(n), S, color="#4C72B0", lw=0.8,
            label="Control statistic S_t")
    ax.axhline(L, color="#C44E52", lw=1.8, linestyle="--",
               label=f"Control limit L={L:.2f}
                   (chi2_{{{rank_S},{alpha_cl:.3f}}})")

    if cp is not None:
        ax.axvline(cp, color="#DD8452", lw=1.5, linestyle=":",
                   label=f"Detected changepoint: t={cp}")
        ax.scatter([cp], [S[cp]], color="#DD8452", s=60, zorder=5)

    ax.set_xlabel("Observation index t", fontsize=11)
    ax.set_ylabel("S_t = (pi_tilde_t - mu)^T Sigma_t^+ (pi_tilde_t - mu)",
                  fontsize=10)
    ax.set_title("EWMA Control Chart - Drone Initial Positions\n"
                 f"(rho={r}, mu=mu_null, rank(Sigma)={rank_S}, L={L:.2f})",
                    fontsize=11)
    ax.legend(fontsize=8, loc="upper left")
    ax.grid(linestyle="--", alpha=0.4)
    fig.tight_layout()

    out_path = OUTPUT_DIR / "q3c_ewma_changepoint.png"
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"EWMA plot saved: {out_path}")

    # Summary
    print(f"\nS_t stats: mean={S.mean():.4f}, max={S.max():.4f}, "
          f"fraction > L: {(S > L).mean():.4f}")

    # Sensitivity analysis for rho
    print("\n-- Rho sensitivity analysis --")
    rho_values = [0.05, 0.1, 0.2, 0.3]
    print(f"  {'rho':>6}  {'L (chi2_0.999)':>16}  {'CP detected':>13}  {'S_t at
        CP':>11}")
    print(f"  {'-'*6}  {'-'*16}  {'-'*13}  {'-'*11}")
    for r_test in rho_values:
        Z_t = mu.copy()
        S_t = np.zeros(n)
        L_t = chi2.ppf(alpha_cl, df=rank_S)
        for i_t in range(1, n):
            rho_factor_t = r_test / (2 - r_test) * (1 - (1 - r_test) ** (2 * i_t))
            Z_t = (1 - r_test) * Z_t + r_test * E[i_t]
            diff_t = Z_t - mu
            S_t[i_t] = float(diff_t @ (Sigma_pinv / rho_factor_t) @ diff_t)
        exc = np.where(S_t > L_t)[0]
        cp_t = int(exc[0]) if len(exc) > 0 else None
        cp_str = str(cp_t) if cp_t is not None else "None"
        s_at_cp = f"{S_t[cp_t]:.3f}" if cp_t is not None else "N/A"
        print(f"  {r_test:>6.2f}  {L_t:>16.4f}  {cp_str:>13}  {s_at_cp:>11}")
        if r_test == rho_values[-1]:
            # Save rho sensitivity plot
            fig_rho, ax_rho = plt.subplots(figsize=(10, 4))

    # Build multi-rho plot
    fig_rho, ax_rho = plt.subplots(figsize=(11, 4))
```

```
    colours_rho = ["#4C72B0", "#DD8452", "#55A868", "#C44E52"]
    for r_test, col_r in zip(rho_values, colours_rho):
        Z_t = mu.copy()
        S_t = np.zeros(n)
        for i_t in range(1, n):
            rho_factor_t = r_test / (2 - r_test) * (1 - (1 - r_test) ** (2 * i_t))
            Z_t = (1 - r_test) * Z_t + r_test * E[i_t]
            diff_t = Z_t - mu
            S_t[i_t] = float(diff_t @ (Sigma_pinv / rho_factor_t) @ diff_t)
        ax_rho.plot(S_t, lw=0.8, color=col_r, alpha=0.8, label=f"rho={r_test}")
    ax_rho.axhline(L, color="black", lw=1.5, linestyle="--",
                   label=f"L={L:.2f}")
    ax_rho.set_xlabel("Observation index t", fontsize=11)
    ax_rho.set_ylabel("S_t", fontsize=11)
    ax_rho.set_title("EWMA sensitivity for rho in {0.05, 0.1, 0.2, 0.3}",
                     fontsize=11)
    ax_rho.legend(fontsize=9, loc="upper left")
    ax_rho.grid(linestyle="--", alpha=0.4)
    fig_rho.tight_layout()
    rho_path = OUTPUT_DIR / "q3c_ewma_rho_sensitivity.png"
    fig_rho.savefig(rho_path, dpi=150)
    plt.close(fig_rho)
    print(f"Rho sensitivity plot saved: {rho_path}")

    return S, L, cp


# -- Main ----------------------------------------------------------------------

def main():
    print("=" * 70)
    print("MATH70099 CW2 - Q3: Markov Mixture + VI + EWMA")
    print("=" * 70)

    print_sufficient_statistics()
    print_cavi_updates()

    # (b) Supplementary synthetic verification of the derived updates
    run_part_b_cavi()

    # (c) EWMA changepoint detection
    run_ewma_changepoint(DRONE_NPY)

    print("\nAll Q3 tasks complete.")


if __name__ == "__main__":
    main()
```

Listing 11: Full source code for Question 3 (`src/q3_markov_vi_ewma.py`)

B.4 Question 4 Source Code

```
"""Question 4: iterative PySpark GLM analysis for EPA Daily AQI."""

from __future__ import annotations

import json
import math
import os
import re
import zipfile
from dataclasses import dataclass
from pathlib import Path
from typing import Any

```

```
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.collections import PatchCollection
from matplotlib.patches import Polygon as MplPolygon
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import GeneralizedLinearRegression
from pyspark.sql import DataFrame, SparkSession, Window
from pyspark.sql import functions as F
from pyspark.storagelevel import StorageLevel


SEED = 6060027
OMEGA = 2.0 * math.pi / 365.25
PROJECT_DIR = Path(__file__).resolve().parent.parent
OUTPUT_DIR = Path(os.environ.get("Q4_OUTPUT_DIR", PROJECT_DIR / "outputs_athena" / "q4"))
DATA_DIR = Path(os.environ.get("Q4_DATA_DIR", PROJECT_DIR / "data" / "q4"))
EPS = 1e-6
US_STATES_GEOJSON = DATA_DIR / "us_states.geojson"

plt.rcParams.update(
    {
        "figure.dpi": 150,
        "font.size": 9,
        "axes.titlesize": 10,
        "axes.labelsize": 9,
        "xtick.labelsize": 8,
        "ytick.labelsize": 8,
        "legend.fontsize": 8,
    }
)
CB = {
    "blue": "#4C72B0",
    "red": "#C44E52",
    "green": "#55A868",
    "orange": "#DD8452",
    "purple": "#8172B2",
    "brown": "#937860",
    "teal": "#64B5CD",
}

STATE_TO_DIVISION = {
    "Connecticut": "New England",
    "Maine": "New England",
    "Massachusetts": "New England",
    "New Hampshire": "New England",
    "Rhode Island": "New England",
    "Vermont": "New England",
    "New Jersey": "Middle Atlantic",
    "New York": "Middle Atlantic",
    "Pennsylvania": "Middle Atlantic",
    "Illinois": "East North Central",
    "Indiana": "East North Central",
    "Michigan": "East North Central",
    "Ohio": "East North Central",
    "Wisconsin": "East North Central",
    "Iowa": "West North Central",
    "Kansas": "West North Central",
    "Minnesota": "West North Central",
    "Missouri": "West North Central",
    "Nebraska": "West North Central",
    "North Dakota": "West North Central",
```

```
    "South Dakota": "West North Central",
    "Delaware": "South Atlantic",
    "District Of Columbia": "South Atlantic",
    "Florida": "South Atlantic",
    "Georgia": "South Atlantic",
    "Maryland": "South Atlantic",
    "North Carolina": "South Atlantic",
    "South Carolina": "South Atlantic",
    "Virginia": "South Atlantic",
    "West Virginia": "South Atlantic",
    "Alabama": "East South Central",
    "Kentucky": "East South Central",
    "Mississippi": "East South Central",
    "Tennessee": "East South Central",
    "Arkansas": "West South Central",
    "Louisiana": "West South Central",
    "Oklahoma": "West South Central",
    "Texas": "West South Central",
    "Arizona": "Mountain",
    "Colorado": "Mountain",
    "Idaho": "Mountain",
    "Montana": "Mountain",
    "Nevada": "Mountain",
    "New Mexico": "Mountain",
    "Utah": "Mountain",
    "Wyoming": "Mountain",
    "Alaska": "Pacific",
    "California": "Pacific",
    "Hawaii": "Pacific",
    "Oregon": "Pacific",
    "Washington": "Pacific",
    "Puerto Rico": "Other",
    "Virgin Islands": "Other",
    "Country Of Mexico": "Other",
}

DIVISION_TO_REGION = {
    "New England": "Northeast",
    "Middle Atlantic": "Northeast",
    "East North Central": "Midwest",
    "West North Central": "Midwest",
    "South Atlantic": "South",
    "East South Central": "South",
    "West South Central": "South",
    "Mountain": "West",
    "Pacific": "West",
    "Other": "Other",
}

DIVISION_ORDER = [
    "New England",
    "Middle Atlantic",
    "East North Central",
    "West North Central",
    "South Atlantic",
    "East South Central",
    "West South Central",
    "Mountain",
    "Pacific",
    "Other",
]
REGION_ORDER = ["Northeast", "Midwest", "South", "West", "Other"]


@dataclass
class CandidateSpec:
```

```
    name: str
    block: str
    description: str
    feature_cols: list[str]
    family: str = "gamma"
    use_positive_only: bool = True
    eligible_for_final: bool = True
    interpretation: str = "structural"
    variance_power: float | None = None
    link_power: float | None = None


def slugify(value: str) -> str:
    text = re.sub(r"[^0-9a-zA-Z]+", "_", value.strip().lower()).strip("_")
    return text or "unknown"


def spark_session() -> SparkSession:
    builder = SparkSession.builder.appName("CW2_Q4_GLM_Search")
    spark_master = os.environ.get("Q4_SPARK_MASTER")
    if spark_master:
        builder = builder.master(spark_master)
    builder = builder.config("spark.sql.execution.arrow.pyspark.enabled", "false")
    builder = builder.config("spark.sql.shuffle.partitions",
        os.environ.get("Q4_SHUFFLE_PARTITIONS", "24"))
    if spark_master and spark_master.startswith("local"):
        builder = builder.config("spark.driver.memory",
            os.environ.get("Q4_DRIVER_MEMORY", "8g"))
        builder = builder.config("spark.default.parallelism",
            os.environ.get("Q4_DEFAULT_PARALLELISM", "24"))
    spark = builder.getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")
    return spark


def load_year(spark: SparkSession, year: int) -> DataFrame:
    csv_path = DATA_DIR / f"daily_aqi_by_county_{year}.csv"
    zip_path = DATA_DIR / f"daily_aqi_by_county_{year}.zip"
    if not csv_path.exists() and zip_path.exists():
        with zipfile.ZipFile(zip_path) as archive:
            members = [name for name in archive.namelist() if
                name.endswith(".csv")]
            archive.extract(members[0], path=DATA_DIR)
            extracted = DATA_DIR / members[0]
            if extracted != csv_path:
                extracted.rename(csv_path)
    return spark.read.option("header", "true").option("inferSchema",
        "false").csv(csv_path.resolve().as_uri())


def load_data(spark: SparkSession) -> DataFrame:
    raw = load_year(spark, 2022).unionByName(load_year(spark, 2023))
    for col in raw.columns:
        clean = col.strip().replace(" ", "_")
        if clean != col:
            raw = raw.withColumnRenamed(col, clean)

    division_map = F.create_map(*[item for pair in STATE_TO_DIVISION.items() for
        item in (F.lit(pair[0]), F.lit(pair[1]))])
    region_map = F.create_map(*[item for pair in DIVISION_TO_REGION.items() for
        item in (F.lit(pair[0]), F.lit(pair[1]))])

    county_name_field = "County_Name" if "County_Name" in raw.columns else
        "county_Name" if "county_Name" in raw.columns else None
    county_name_col = F.col(county_name_field) if county_name_field else
        F.lit("Unknown")
```

```
    county_code_col = F.lpad(F.coalesce(F.col("County_Code"), F.lit("")), 3, "0")
        if "County_Code" in raw.columns else F.lit("")
    state_code_col = F.lpad(F.coalesce(F.col("State_Code"), F.lit("")), 2, "0")
        if "State_Code" in raw.columns else F.lit("")

    df = (
        raw.withColumn("AQI", F.col("AQI").cast("double"))
        .withColumn("Number_of_Sites_Reporting",
            F.col("Number_of_Sites_Reporting").cast("double"))
        .filter(F.col("AQI").isNotNull() & F.col("Date").isNotNull() &
            F.col("State_Name").isNotNull())
        .withColumn("date_parsed", F.to_date("Date", "yyyy-MM-dd"))
        .filter(F.col("date_parsed").isNotNull())
        .withColumn("year", F.year("date_parsed").cast("int"))
        .withColumn("month", F.month("date_parsed").cast("int"))
        .withColumn("day_of_year", F.dayofyear("date_parsed").cast("int"))
        .withColumn("division", F.coalesce(division_map[F.col("State_Name")],
            F.lit("Other")))
        .withColumn("region", F.coalesce(region_map[F.col("division")],
            F.lit("Other")))
        .withColumn("county_name_clean", F.coalesce(county_name_col,
            F.lit("Unknown")))
        .withColumn(
            "county_id",
            F.when(
                (F.length(F.trim(state_code_col)) > 0) &
                    (F.length(F.trim(county_code_col)) > 0),
                F.concat_ws("-", state_code_col, county_code_col),
            ).otherwise(F.concat_ws("-", F.col("State_Name"),
                F.col("county_name_clean"))),
        )
        .withColumn("Defining_Parameter", F.coalesce(F.col("Defining_Parameter"),
            F.lit("Unknown")))
        .withColumn("Category", F.coalesce(F.col("Category"), F.lit("Unknown")))
        .withColumn("log_sites",
            F.log1p(F.coalesce(F.col("Number_of_Sites_Reporting"), F.lit(0.0))))
        .persist(StorageLevel.DISK_ONLY)
    )
    df.count()
    return df


def collect_levels(df: DataFrame, col_name: str, preferred_baseline: str | None =
    None, preferred_order: list[str] | None = None) -> tuple[str, list[str]]:
    values = [row[0] for row in
        df.select(col_name).distinct().orderBy(col_name).collect()]
    ordered = []
    if preferred_order:
        ordered.extend([value for value in preferred_order if value in values])
    ordered.extend([value for value in values if value not in ordered])
    baseline = preferred_baseline if preferred_baseline in ordered else ordered[0]
    kept = [value for value in ordered if value != baseline]
    return baseline, kept


def add_dummy_columns(df: DataFrame, source_col: str, levels: list[str], prefix:
    str) -> tuple[DataFrame, dict[str, str]]:
    columns = {}
    for level in levels:
        name = f"{prefix}_{slugify(level)}"
        df = df.withColumn(name, (F.col(source_col) ==
            F.lit(level)).cast("double"))
        columns[level] = name
    return df, columns


```

```python
def add_interactions(df: DataFrame, left_map: dict[str, str], right_cols: list[str], prefix: str) -> tuple[DataFrame, list[str]]:
    out_cols = []
    for level, left_col in left_map.items():
        level_slug = slugify(level)
        for right_col in right_cols:
            name = f"{prefix}_{level_slug}_{right_col}"
            df = df.withColumn(name, F.col(left_col) * F.col(right_col))
            out_cols.append(name)
    return df, out_cols


def add_train_only_history_features(df: DataFrame) -> tuple[DataFrame, dict[str, float]]:
    train = df.filter(F.col("year") == 2022)
    overall = train.agg(F.mean("AQI").alias("aqi_mean")).first().asDict()
    county_stats = (
        train.groupBy("county_id")
        .agg(
            F.mean("AQI").alias("county_mean_aqi_2022"),
            F.mean(F.when(F.col("month").between(6, 9), F.col("AQI"))).alias("county_summer_mean_2022"),
            F.mean((F.col("Defining_Parameter") == "PM2.5").cast("double")).alias("county_pm25_share_2022"),
        )
        .withColumn(
            "county_summer_mean_2022",
            F.coalesce(F.col("county_summer_mean_2022"), F.col("county_mean_aqi_2022")),
        )
    )
    defaults = county_stats.agg(
        F.mean("county_mean_aqi_2022").alias("county_mean_aqi_2022"),
        F.mean("county_summer_mean_2022").alias("county_summer_mean_2022"),
        F.mean("county_pm25_share_2022").alias("county_pm25_share_2022"),
    ).first().asDict()

    enriched = (
        df.join(county_stats, on="county_id", how="left")
        .fillna(defaults)
        .withColumn("county_log_mean", F.log1p(F.col("county_mean_aqi_2022")))
        .withColumn(
            "county_summer_uplift",
            F.log1p(F.col("county_summer_mean_2022")) - F.log1p(F.col("county_mean_aqi_2022")),
        )
    )
    overall["county_mean_default"] = float(defaults["county_mean_aqi_2022"])
    overall["county_summer_default"] = float(defaults["county_summer_mean_2022"])
    return enriched, {key: float(value) for key, value in overall.items()}


def add_temporal_features(df: DataFrame, fallback_value: float) -> DataFrame:
    date_index = F.datediff(F.col("date_parsed"), F.lit("1970-01-01"))
    lag_window = Window.partitionBy("county_id").orderBy("date_index")
    roll_window = lag_window.rangeBetween(-7, -1)
    county_fallback = F.coalesce(F.col("county_mean_aqi_2022"), F.lit(fallback_value))
    return (
        df.withColumn("date_index", date_index)
        .withColumn("lag1_aqi", F.lag("AQI", 1).over(lag_window))
        .withColumn("lag7_aqi", F.lag("AQI", 7).over(lag_window))
        .withColumn("lag30_aqi", F.lag("AQI", 30).over(lag_window))
        .withColumn("roll7_mean_aqi", F.avg("AQI").over(roll_window))
        .withColumn("lag1_aqi_filled", F.coalesce(F.col("lag1_aqi"), county_fallback))
```

```
        .withColumn("lag7_aqi_filled", F.coalesce(F.col("lag7_aqi"),
            county_fallback))
        .withColumn("lag30_aqi_filled", F.coalesce(F.col("lag30_aqi"),
            county_fallback))
        .withColumn("roll7_mean_aqi_filled", F.coalesce(F.col("roll7_mean_aqi"),
            county_fallback))
        .withColumn("log1p_lag1_aqi", F.log1p(F.col("lag1_aqi_filled")))
        .withColumn("log1p_lag7_aqi", F.log1p(F.col("lag7_aqi_filled")))
        .withColumn("log1p_lag30_aqi", F.log1p(F.col("lag30_aqi_filled")))
        .withColumn("log1p_roll7_mean_aqi",
            F.log1p(F.col("roll7_mean_aqi_filled")))
    )


def prepare_feature_bank(df: DataFrame) -> tuple[DataFrame, dict[str, Any]]:
    df, history_defaults = add_train_only_history_features(df)
    df = add_temporal_features(df, history_defaults["aqi_mean"])
    df = (
        df.withColumn("sin1", F.sin(F.lit(OMEGA) * F.col("day_of_year")))
        .withColumn("cos1", F.cos(F.lit(OMEGA) * F.col("day_of_year")))
        .withColumn("sin2", F.sin(F.lit(2.0 * OMEGA) * F.col("day_of_year")))
        .withColumn("cos2", F.cos(F.lit(2.0 * OMEGA) * F.col("day_of_year")))
    )

    train = df.filter(F.col("year") == 2022)
    region_baseline, region_levels = collect_levels(train, "region",
        preferred_baseline="Northeast", preferred_order=REGION_ORDER)
    division_baseline, division_levels = collect_levels(
        train,
        "division",
        preferred_baseline="New England",
        preferred_order=DIVISION_ORDER,
    )
    pollutant_baseline, pollutant_levels = collect_levels(train,
        "Defining_Parameter", preferred_baseline="Ozone")

    df, region_dummy_map = add_dummy_columns(df, "region", region_levels,
        "region")
    df, division_dummy_map = add_dummy_columns(df, "division", division_levels,
        "division")
    df, pollutant_dummy_map = add_dummy_columns(df, "Defining_Parameter",
        pollutant_levels, "poll")
    df, region_season_cols = add_interactions(df, region_dummy_map, ["sin1",
        "cos1"], "region_season")
    df, pollutant_season_cols = add_interactions(df, pollutant_dummy_map,
        ["sin1", "cos1"], "poll_season")
    structural_feature_cols = (
        ["sin1", "cos1", "sin2", "cos2"]
        + list(division_dummy_map.values())
        + list(pollutant_dummy_map.values())
        + ["log_sites", "county_log_mean", "county_summer_uplift",
            "county_pm25_share_2022"]
        + pollutant_season_cols
        + region_season_cols
    )
    temporal_feature_cols = [
        "log1p_lag1_aqi",
        "log1p_lag7_aqi",
        "log1p_lag30_aqi",
        "log1p_roll7_mean_aqi",
    ]

    metadata = {
        "region_baseline": region_baseline,
        "division_baseline": division_baseline,
        "pollutant_baseline": pollutant_baseline,
```

```
        "region_dummy_cols": list(region_dummy_map.values()),
        "division_dummy_cols": list(division_dummy_map.values()),
        "pollutant_dummy_cols": list(pollutant_dummy_map.values()),
        "region_season_cols": region_season_cols,
        "pollutant_season_cols": pollutant_season_cols,
        "history_defaults": history_defaults,
        "region_levels": region_levels,
        "division_levels": division_levels,
        "pollutant_levels": pollutant_levels,
        "structural_feature_cols": structural_feature_cols,
        "temporal_feature_cols": temporal_feature_cols,
    }
    df = df.persist(StorageLevel.DISK_ONLY)
    df.count()
    return df, metadata


def gamma_deviance_expr(label_col: str, pred_col: str) -> F.Column:
    label = F.col(label_col)
    pred = F.col(pred_col)
    return 2.0 * (((label - pred) / pred) - F.log(label / pred))


def tweedie_deviance_expr(label_col: str, pred_col: str, variance_power: float)
    -> F.Column:
    label = F.col(label_col)
    pred = F.col(pred_col)
    p = variance_power
    first = F.when(label > 0, F.pow(label, 2.0 - p) / ((1.0 - p) * (2.0 -
        p))).otherwise(F.lit(0.0))
    second = -(label * F.pow(pred, 1.0 - p)) / (1.0 - p)
    third = F.pow(pred, 2.0 - p) / (2.0 - p)
    return 2.0 * (first + second + third)


def fit_glm(train_df: DataFrame, spec: CandidateSpec):
    assembler = VectorAssembler(inputCols=spec.feature_cols, outputCol="features")
    train_vec = assembler.transform(train_df).select("AQI", "features")
    kwargs: dict[str, Any] = {
        "featuresCol": "features",
        "labelCol": "AQI",
        "predictionCol": "prediction",
        "family": spec.family,
        "fitIntercept": True,
        "maxIter": 120,
        "regParam": 0.0,
        "solver": "irls",
    }
    if spec.family == "gamma":
        kwargs["link"] = "log"
    if spec.family == "tweedie":
        kwargs["variancePower"] = spec.variance_power if spec.variance_power is
            not None else 1.5
        kwargs["linkPower"] = spec.link_power if spec.link_power is not None else
            0.0
    model = GeneralizedLinearRegression(**kwargs).fit(train_vec)
    return model, assembler


def evaluate_model(model, assembler: VectorAssembler, test_df: DataFrame, spec:
    CandidateSpec) -> dict[str, Any]:
    pred_df = (
        model.transform(assembler.transform(test_df))
        .withColumn("prediction", F.greatest(F.col("prediction"), F.lit(EPS)))
        .persist(StorageLevel.DISK_ONLY)
    )
```

```
    positive_eval = pred_df.filter(F.col("AQI") > 0)
    common_metrics = positive_eval.agg(
        F.count("*").alias("n_eval"),
        F.avg(F.abs(F.col("AQI") - F.col("prediction"))).alias("mae"),
        F.sqrt(F.avg(F.pow(F.col("AQI") - F.col("prediction"),
            2.0))).alias("rmse"),
        F.avg(gamma_deviance_expr("AQI",
            "prediction")).alias("mean_gamma_deviance"),
        F.corr("AQI", "prediction").alias("corr"),
    ).first().asDict()

    metrics: dict[str, Any] = {key: (float(value) if value is not None else
        math.nan) for key, value in common_metrics.items()}
    metrics["family"] = spec.family
    metrics["uses_zero_aqi"] = not spec.use_positive_only
    metrics["eligible_for_final"] = spec.eligible_for_final
    metrics["interpretation"] = spec.interpretation
    metrics["feature_count"] = len(spec.feature_cols)
    metrics["block"] = spec.block
    metrics["description"] = spec.description
    metrics["model_name"] = spec.name

    all_metrics = pred_df.agg(
        F.count("*").alias("n_eval_all"),
        F.avg(F.abs(F.col("AQI") - F.col("prediction"))).alias("mae_all"),
        F.sqrt(F.avg(F.pow(F.col("AQI") - F.col("prediction"),
            2.0))).alias("rmse_all"),
    ).first().asDict()
    for key, value in all_metrics.items():
        metrics[key] = float(value)

    if spec.family == "tweedie":
        tweedie_mean = pred_df.agg(
            F.avg(tweedie_deviance_expr("AQI", "prediction", spec.variance_power
                if spec.variance_power is not None else 1.5)).alias(
                "mean_tweedie_deviance"
            )
        ).first()["mean_tweedie_deviance"]
        metrics["mean_tweedie_deviance"] = float(tweedie_mean)
    else:
        metrics["mean_tweedie_deviance"] = math.nan

    try:
        summary = model.summary
        metrics["aic"] = float(summary.aic)
        metrics["dispersion"] = float(summary.dispersion)
        metrics["deviance"] = float(summary.deviance)
        metrics["null_deviance"] = float(summary.nullDeviance)
    except Exception:
        metrics["aic"] = math.nan
        metrics["dispersion"] = math.nan
        metrics["deviance"] = math.nan
        metrics["null_deviance"] = math.nan
    pred_df.unpersist()
    return metrics


def null_model_metrics(train_df: DataFrame, test_df: DataFrame) -> dict[str, Any]:
    train_positive = train_df.filter(F.col("AQI") > 0)
    test_positive = test_df.filter(F.col("AQI") > 0)
    mean_pred =
        float(train_positive.agg(F.mean("AQI").alias("mean_aqi")).first()["mean_aqi"])
    pred_df = (
        test_df.select("AQI")
        .withColumn("prediction", F.lit(mean_pred))
        .withColumn("prediction", F.greatest(F.col("prediction"), F.lit(EPS)))
```

```
    )
    positive_eval = pred_df.filter(F.col("AQI") > 0)
    common = positive_eval.agg(
        F.count("*").alias("n_eval"),
        F.avg(F.abs(F.col("AQI") - F.col("prediction"))).alias("mae"),
        F.sqrt(F.avg(F.pow(F.col("AQI") - F.col("prediction"),
            2.0))).alias("rmse"),
        F.avg(gamma_deviance_expr("AQI",
            "prediction")).alias("mean_gamma_deviance"),
    ).first().asDict()
    all_metrics = pred_df.agg(
        F.count("*").alias("n_eval_all"),
        F.avg(F.abs(F.col("AQI") - F.col("prediction"))).alias("mae_all"),
        F.sqrt(F.avg(F.pow(F.col("AQI") - F.col("prediction"),
            2.0))).alias("rmse_all"),
    ).first().asDict()
    result = {key: float(value) for key, value in common.items()}
    result.update({key: float(value) for key, value in all_metrics.items()})
    result.update(
        {
            "corr": math.nan,
            "family": "null",
            "uses_zero_aqi": True,
            "eligible_for_final": False,
            "interpretation": "benchmark",
            "feature_count": 0,
            "block": "reference",
            "description": "2022 positive-AQI mean",
            "model_name": "null_mean",
            "mean_tweedie_deviance": math.nan,
            "aic": math.nan,
            "dispersion": math.nan,
            "deviance": math.nan,
            "null_deviance": math.nan,
        }
    )
    return result


def coefficient_table(model, feature_cols: list[str]) -> pd.DataFrame:
    summary = model.summary
    estimates = [float(model.intercept)] + [float(value) for value in
        model.coefficients.toArray().tolist()]
    std_err = [float(value) for value in summary.coefficientStandardErrors]
    p_values = [float(value) for value in summary.pValues]
    ci_low = [estimate - 1.96 * se for estimate, se in zip(estimates, std_err)]
    ci_high = [estimate + 1.96 * se for estimate, se in zip(estimates, std_err)]
    terms = ["intercept"] + feature_cols
    return pd.DataFrame(
        {
            "term": terms,
            "estimate": estimates,
            "std_error": std_err,
            "p_value": p_values,
            "aqi_ratio": np.exp(estimates),
            "ci_low": np.exp(ci_low),
            "ci_high": np.exp(ci_high),
        }
    )


def collect_data_summary(df: DataFrame, metadata: dict[str, Any]) -> dict[str,
    Any]:
    quantiles = [float(value) for value in df.approxQuantile("AQI", [0.25, 0.5,
        0.75], 0.0)]
    monthly = (
```

```
        df.groupBy("month")
        .agg(
            F.mean("AQI").alias("mean_aqi"),
            F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
        )
        .orderBy("month")
        .toPandas()
    )
    region_summary = (
        df.groupBy("region")
        .agg(
            F.count("*").alias("n_obs"),
            F.mean("AQI").alias("mean_aqi"),
            F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
        )
        .orderBy(F.desc("mean_aqi"))
        .toPandas()
    )
    pollutant_summary = (
        df.groupBy("Defining_Parameter")
        .agg(
            F.count("*").alias("n_obs"),
            F.mean("AQI").alias("mean_aqi"),
            F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
        )
        .orderBy(F.desc("unhealthy_rate"), F.desc("n_obs"))
        .toPandas()
    )
    summary = df.agg(
        F.count("*").alias("n_total"),
        F.sum((F.col("AQI") > 0).cast("double")).alias("n_positive"),
        F.sum((F.col("AQI") == 0).cast("double")).alias("n_zero"),
        F.sum((F.col("AQI") > 100).cast("double")).alias("unhealthy_count"),
        F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
        F.mean("AQI").alias("aqi_mean"),
    ).first().asDict()
    return {
        **{key: float(value) for key, value in summary.items()},
        "aqi_quantiles": [int(round(value)) for value in quantiles],
        "monthly_mean_aqi": {str(int(row["month"])): float(row["mean_aqi"]) for
            _, row in monthly.iterrows()},
        "monthly_unhealthy_rate": {str(int(row["month"])):
            float(row["unhealthy_rate"]) for _, row in monthly.iterrows()},
        "region_summary": [
            {
                "region": str(row["region"]),
                "n_obs": int(row["n_obs"]),
                "mean_aqi": float(row["mean_aqi"]),
                "unhealthy_rate": float(row["unhealthy_rate"]),
            }
            for _, row in region_summary.iterrows()
        ],
        "pollutant_summary": [
            {
                "defining_parameter": str(row["Defining_Parameter"]),
                "n_obs": int(row["n_obs"]),
                "mean_aqi": float(row["mean_aqi"]),
                "unhealthy_rate": float(row["unhealthy_rate"]),
            }
            for _, row in pollutant_summary.iterrows()
        ],
        "region_baseline": metadata["region_baseline"],
        "division_baseline": metadata["division_baseline"],
        "pollutant_baseline": metadata["pollutant_baseline"],
    }

```

```
610
611 def plot_eda(df: DataFrame) -> None:
612     sample_pdf = df.select("AQI").sample(False, 0.03, seed=SEED).toPandas()
613     monthly_national_pdf = (
614         df.groupBy("month")
615         .agg(
616             F.mean("AQI").alias("mean_aqi"),
617             F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
618         )
619         .orderBy("month")
620         .toPandas()
621     )
622     region_summary_pdf = (
623         df.groupBy("region")
624         .agg(
625             F.count("*").alias("n_obs"),
626             F.mean("AQI").alias("mean_aqi"),
627             F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
628         )
629         .orderBy(F.desc("unhealthy_rate"), F.desc("mean_aqi"))
630         .toPandas()
631     )
632     monthly_pdf = (
633         df.groupBy("region", "month")
634         .agg(
635             F.mean("AQI").alias("mean_aqi"),
636             F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
637         )
638         .toPandas()
639     )
640     pollutant_pdf = (
641         df.groupBy("Defining_Parameter")
642         .agg(
643             F.count("*").alias("n_obs"),
644             F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
645         )
646         .orderBy(F.desc("unhealthy_rate"), F.desc("n_obs"))
647         .toPandas()
648     )
649     state_pdf = (
650         df.groupBy("State_Name")
651         .agg(
652             F.count("*").alias("n_obs"),
653             F.sum((F.col("AQI") > 100).cast("double")).alias("unhealthy_days"),
654             F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
655             F.mean("AQI").alias("mean_aqi"),
656         )
657         .filter(F.col("unhealthy_days") > 0)
658         .orderBy(F.desc("unhealthy_days"), F.desc("unhealthy_rate"))
659         .limit(10)
660         .toPandas()
661     )
662
663     fig, axes = plt.subplots(3, 2, figsize=(10.6, 10.0))
664     axes = axes.ravel()
665     axes[0].hist(sample_pdf["AQI"].to_numpy(), bins=80, density=True,
        color=CB["blue"], alpha=0.85, edgecolor="none")
666     axes[0].axvline(100, color=CB["red"], linewidth=1.8, linestyle="--",
        label="AQI = 100")
667     axes[0].set_xlim(0, 250)
668     axes[0].set_xlabel("AQI")
669     axes[0].set_ylabel("Density")
670     axes[0].set_title("(a) Daily AQI distribution")
671     axes[0].legend(frameon=False)
672
673     ax_month = axes[1]
```

```
    month_values = monthly_national_pdf["month"].to_numpy()
    ax_month.bar(month_values, monthly_national_pdf["mean_aqi"].to_numpy(),
        color=CB["blue"], alpha=0.55, label="Mean AQI")
    ax_month.set_xticks(range(1, 13))
    ax_month.set_xlabel("Month")
    ax_month.set_ylabel("Mean AQI")
    ax_month.set_title("(b) National seasonality")
    ax_month.grid(alpha=0.25, linestyle="--", axis="y")
    ax_month_rate = ax_month.twinx()
    ax_month_rate.plot(
        month_values,
        100.0 * monthly_national_pdf["unhealthy_rate"].to_numpy(),
        color=CB["red"],
        marker="o",
        linewidth=1.8,
        markersize=3.5,
        label="Unhealthy rate (%)",
    )
    ax_month_rate.set_ylabel("Unhealthy rate (%)")
    lines1, labels1 = ax_month.get_legend_handles_labels()
    lines2, labels2 = ax_month_rate.get_legend_handles_labels()
    ax_month.legend(lines1 + lines2, labels1 + labels2, frameon=False, loc="upper
        left")
    ax_month_rate.set_ylim(bottom=0)

    palette = {"Northeast": CB["blue"], "Midwest": CB["green"], "South":
        CB["orange"], "West": CB["red"], "Other": CB["brown"]}
    for region, region_pdf in monthly_pdf.groupby("region"):
        region_pdf = region_pdf.sort_values("month")
        axes[2].plot(
            region_pdf["month"].to_numpy(),
            region_pdf["mean_aqi"].to_numpy(),
            marker="o",
            linewidth=1.8,
            markersize=3.5,
            label=region,
            color=palette.get(region, CB["purple"]),
        )
    axes[2].set_xticks(range(1, 13))
    axes[2].set_xlabel("Month")
    axes[2].set_ylabel("Mean AQI")
    axes[2].set_title("(c) Monthly mean AQI by region")
    axes[2].legend(frameon=False, ncol=2)
    axes[2].grid(alpha=0.25, linestyle="--")

    region_plot = region_summary_pdf.sort_values("unhealthy_rate", ascending=True)
    axes[3].barh(
        region_plot["region"].astype(str),
        100.0 * region_plot["unhealthy_rate"].to_numpy(),
        color=CB["teal"],
        alpha=0.85,
    )
    axes[3].set_xlabel("Unhealthy rate (%)")
    axes[3].set_title("(d) Unhealthy-day rate by region")
    axes[3].grid(alpha=0.25, linestyle="--", axis="x")

    pollutant_plot = pollutant_pdf.sort_values("unhealthy_rate", ascending=True)
    axes[4].barh(
        pollutant_plot["Defining_Parameter"].astype(str),
        100.0 * pollutant_plot["unhealthy_rate"].to_numpy(),
        color=CB["orange"],
        alpha=0.8,
    )
    axes[4].set_xlabel("Unhealthy rate (%)")
    axes[4].set_title("(e) Unhealthy-day rate by defining pollutant")
    axes[4].grid(alpha=0.25, linestyle="--", axis="x")
```

```
    state_plot = state_pdf.sort_values("unhealthy_days", ascending=True)
    axes[5].barh(
        state_plot["State_Name"].astype(str),
        state_plot["unhealthy_days"].to_numpy(),
        color=CB["purple"],
        alpha=0.85,
    )
    axes[5].set_xlabel("Unhealthy county-days")
    axes[5].set_title("(f) Top states by unhealthy-day count")
    axes[5].grid(alpha=0.25, linestyle="--", axis="x")

    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / "q4_eda.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    monthly_pdf.to_csv(OUTPUT_DIR / "q4_region_month_summary.csv", index=False)
    pollutant_pdf.to_csv(OUTPUT_DIR / "q4_pollutant_summary.csv", index=False)
    region_summary_pdf.to_csv(OUTPUT_DIR / "q4_region_summary.csv", index=False)
    state_pdf.to_csv(OUTPUT_DIR / "q4_top_states_unhealthy.csv", index=False)


def plot_state_map(df: DataFrame) -> None:
    if not US_STATES_GEOJSON.exists():
        print(f"Skipping state map: {US_STATES_GEOJSON} not found")
        return

    state_summary_pdf = (
        df.groupBy("State_Name")
        .agg(
            F.count("*").alias("n_obs"),
            F.mean("AQI").alias("mean_aqi"),
            F.sum((F.col("AQI") > 100).cast("double")).alias("unhealthy_days"),
            F.mean((F.col("AQI") > 100).cast("double")).alias("unhealthy_rate"),
        )
        .filter(~F.col("State_Name").isin("Country Of Mexico", "Virgin Islands",
            "Puerto Rico"))
        .toPandas()
    )
    state_summary_pdf.to_csv(OUTPUT_DIR / "q4_state_summary.csv", index=False)
    value_map = {
        str(row["State_Name"]): float(row["unhealthy_rate"])
        for _, row in state_summary_pdf.iterrows()
    }

    with US_STATES_GEOJSON.open("r", encoding="utf-8") as handle:
        geojson = json.load(handle)

    values = np.array(list(value_map.values()))
    vmin = float(np.nanmin(values))
    vmax = float(np.nanmax(values))
    norm = plt.Normalize(vmin=vmin, vmax=vmax)
    cmap = plt.cm.YlOrRd

    fig, ax = plt.subplots(figsize=(11.2, 7.2))
    patches = []
    colors = []

    def add_polygon(coords: list[list[float]], value: float) -> None:
        xy = np.asarray(coords, dtype=float)
        if xy.ndim == 2 and xy.shape[1] >= 2:
            patches.append(MplPolygon(xy[:, :2], closed=True))
            colors.append(value)

    for feature in geojson.get("features", []):
        state_name = feature.get("properties", {}).get("name")
```

```
        if state_name not in value_map:
            continue
        if state_name in {"Alaska", "Hawaii"}:
            continue
        geom = feature.get("geometry", {})
        geom_type = geom.get("type")
        coords = geom.get("coordinates", [])
        value = value_map[state_name]

        if geom_type == "Polygon":
            for ring in coords[:1]:
                add_polygon(ring, value)
        elif geom_type == "MultiPolygon":
            for poly in coords:
                for ring in poly[:1]:
                    add_polygon(ring, value)

    collection = PatchCollection(
        patches,
        cmap=cmap,
        norm=norm,
        edgecolor="white",
        linewidth=0.45,
    )
    collection.set_array(np.asarray(colors))
    ax.add_collection(collection)
    ax.set_xlim(-125, -66)
    ax.set_ylim(24, 50)
    ax.set_aspect("equal", adjustable="box")
    ax.axis("off")
    ax.set_title("State-Level Unhealthy AQI Rate (2022-2023)", fontsize=12)
    cbar = fig.colorbar(collection, ax=ax, fraction=0.03, pad=0.02)
    cbar.set_label("Unhealthy-day rate")

    top_states = state_summary_pdf.sort_values("unhealthy_rate",
        ascending=False).head(5)
    note = "\n".join(
        f"{row['State_Name']}: {100.0 * row['unhealthy_rate']:.2f}%"
        for _, row in top_states.iterrows()
    )
    ax.text(
        -124.5,
        24.7,
        "Top unhealthy-day rates\n" + note,
        fontsize=8,
        va="bottom",
        ha="left",
        bbox={"facecolor": "white", "alpha": 0.85, "edgecolor": "none", "pad": 4},
    )

    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / "q4_state_unhealthy_map.png", dpi=220,
        bbox_inches="tight")
    plt.close(fig)


def plot_diagnostics(pred_pdf: pd.DataFrame) -> None:
    pred_pdf = pred_pdf.copy()
    pred_pdf["decile"] = pd.qcut(pred_pdf["prediction"], 10, labels=False,
        duplicates="drop")
    calib = pred_pdf.groupby("decile", as_index=False).agg(
        fitted=("prediction", "mean"),
        observed=("AQI", "mean"),
    )
    monthly = pred_pdf.groupby(["month"], as_index=False).agg(
        observed=("AQI", "mean"),
```

```
        fitted=("prediction", "mean"),
    )

    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6))
    low = min(calib["fitted"].min(), calib["observed"].min())
    high = max(calib["fitted"].max(), calib["observed"].max())
    axes[0].scatter(calib["fitted"].to_numpy(), calib["observed"].to_numpy(),
        s=28, color=CB["blue"])
    axes[0].plot([low, high], [low, high], color=CB["red"], linestyle="--",
        linewidth=1.4)
    axes[0].set_xlabel("Mean fitted AQI by decile")
    axes[0].set_ylabel("Mean observed AQI by decile")
    axes[0].set_title("(a) Calibration on 2023 deciles")

    axes[1].plot(monthly["month"].to_numpy(), monthly["observed"].to_numpy(),
        marker="o", linewidth=1.8, color=CB["red"], label="Observed")
    axes[1].plot(monthly["month"].to_numpy(), monthly["fitted"].to_numpy(),
        marker="o", linewidth=1.8, color=CB["blue"], label="Fitted")
    axes[1].set_xticks(range(1, 13))
    axes[1].set_xlabel("Month")
    axes[1].set_ylabel("Mean AQI")
    axes[1].set_title("(b) 2023 monthly means")
    axes[1].legend(frameon=False)
    axes[1].grid(alpha=0.25, linestyle="--")

    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / "q4_gamma_diagnostic.png", dpi=220,
        bbox_inches="tight")
    plt.close(fig)


def plot_effects(pred_pdf: pd.DataFrame, coef_pdf: pd.DataFrame, selected_spec:
    CandidateSpec) -> dict[str, float]:
    effect_terms = coef_pdf[coef_pdf["term"].isin(["county_log_mean",
        "county_summer_uplift"])].set_index("term")
    county_quantiles = pred_pdf["county_mean_aqi_2022"].quantile([0.25,
        0.75]).to_dict()
    historical_ratio = math.nan
    if "county_log_mean" in effect_terms.index:
        delta = math.log1p(county_quantiles[0.75]) -
            math.log1p(county_quantiles[0.25])
        historical_ratio = float(math.exp(effect_terms.loc["county_log_mean",
            "estimate"] * delta))

    pred_pdf = pred_pdf.copy()
    pred_pdf["county_decile"] = pd.qcut(pred_pdf["county_mean_aqi_2022"], 10,
        labels=False, duplicates="drop")
    county_curve = pred_pdf.groupby("county_decile", as_index=False).agg(
        historical_mean=("county_mean_aqi_2022", "mean"),
        observed=("AQI", "mean"),
        fitted=("prediction", "mean"),
    )

    keep = coef_pdf[
        coef_pdf["term"].str.startswith("poll_")
        | coef_pdf["term"].str.startswith("division_")
        | coef_pdf["term"].isin(["county_log_mean", "county_summer_uplift"])
    ].copy()
    keep["plot_label"] = (
        keep["term"]
        .str.replace("poll_", "poll: ", regex=False)
        .str.replace("division_", "div: ", regex=False)
        .str.replace("_", " ", regex=False)
    )
    keep = keep.sort_values("estimate")

```

```
    fig, axes = plt.subplots(1, 2, figsize=(10.4, 4.0))
    axes[0].plot(
        county_curve["historical_mean"].to_numpy(),
        county_curve["observed"].to_numpy(),
        marker="o",
        linewidth=1.8,
        color=CB["red"],
        label="Observed",
    )
    axes[0].plot(
        county_curve["historical_mean"].to_numpy(),
        county_curve["fitted"].to_numpy(),
        marker="o",
        linewidth=1.8,
        color=CB["blue"],
        label="Fitted",
    )
    axes[0].set_xlabel("2022 county mean AQI decile midpoint")
    axes[0].set_ylabel("Mean 2023 AQI")
    axes[0].set_title("(a) Historical county baseline signal")
    axes[0].legend(frameon=False)
    axes[0].grid(alpha=0.25, linestyle="--")

    axes[1].errorbar(
        keep["aqi_ratio"],
        keep["plot_label"].to_numpy(),
        xerr=[(keep["aqi_ratio"] - keep["ci_low"]).to_numpy(), (keep["ci_high"] -
            keep["aqi_ratio"]).to_numpy()],
        fmt="o",
        color=CB["blue"],
        ecolor=CB["blue"],
        capsize=2,
    )
    axes[1].axvline(1.0, color=CB["red"], linestyle="--", linewidth=1.2)
    axes[1].set_xlabel("AQI ratio")
    axes[1].set_title(f"(b) Selected {selected_spec.family.title()}-GLM effects")

    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / "q4_effects.png", dpi=220, bbox_inches="tight")
    plt.close(fig)
    return {"county_iqr_ratio": historical_ratio}


def candidate_specs(metadata: dict[str, Any]) -> list[CandidateSpec]:
    structural_cols = metadata["structural_feature_cols"]
    temporal_cols = metadata["temporal_feature_cols"]
    return [
        CandidateSpec(
            name="gamma_structural",
            block="reference",
            description="Structural Gamma GLM with seasonal harmonics, division
                and pollutant effects, county history, and region-season
                interactions",
            feature_cols=structural_cols,
        ),
        CandidateSpec(
            name="gamma_hybrid_temporal",
            block="C",
            description="Hybrid Gamma GLM adding county-wise lagged AQI and a
                backward seven-day rolling mean",
            feature_cols=structural_cols + temporal_cols,
            interpretation="hybrid_prediction",
        ),
    ]


```

```
def fit_all_models(feature_df: DataFrame, metadata: dict[str, Any]) ->
    tuple[pd.DataFrame, dict[str, Any]]:
    train_all = feature_df.filter(F.col("year") ==
        2022).persist(StorageLevel.DISK_ONLY)
    test_all = feature_df.filter(F.col("year") ==
        2023).persist(StorageLevel.DISK_ONLY)
    train_pos = train_all.filter(F.col("AQI") > 0).persist(StorageLevel.DISK_ONLY)
    test_pos = test_all.filter(F.col("AQI") > 0).persist(StorageLevel.DISK_ONLY)
    train_pos.count()
    test_pos.count()

    results = [null_model_metrics(train_all, test_all)]
    spec_map = {spec.name: spec for spec in candidate_specs(metadata)}
    for spec in spec_map.values():
        train_df = train_pos if spec.use_positive_only else train_all
        test_df = test_pos if spec.use_positive_only else test_all
        try:
            model, assembler = fit_glm(train_df, spec)
            metrics = evaluate_model(model, assembler, test_df, spec)
            results.append(metrics)
            print(
                f"{spec.name}: gamma
                    deviance={metrics['mean_gamma_deviance']:.4f}, "
                f"rmse={metrics['rmse']:.2f}, corr={metrics['corr']:.3f}"
            )
        except Exception as exc:
            results.append(
                {
                    "model_name": spec.name,
                    "block": spec.block,
                    "description": spec.description,
                    "family": spec.family,
                    "uses_zero_aqi": not spec.use_positive_only,
                    "eligible_for_final": spec.eligible_for_final,
                    "interpretation": spec.interpretation,
                    "feature_count": len(spec.feature_cols),
                    "n_eval": math.nan,
                    "mae": math.nan,
                    "rmse": math.nan,
                    "mean_gamma_deviance": math.nan,
                    "corr": math.nan,
                    "n_eval_all": math.nan,
                    "mae_all": math.nan,
                    "rmse_all": math.nan,
                    "mean_tweedie_deviance": math.nan,
                    "aic": math.nan,
                    "dispersion": math.nan,
                    "deviance": math.nan,
                    "null_deviance": math.nan,
                    "error": str(exc),
                }
            )
            print(f"{spec.name} failed: {exc}")

    comparison = pd.DataFrame(results)
    structural_row = comparison.loc[comparison["model_name"] ==
        "gamma_structural"].iloc[0].to_dict()
    hybrid_row = comparison.loc[comparison["model_name"] ==
        "gamma_hybrid_temporal"].iloc[0].to_dict()
    deviance_gain = float(structural_row["mean_gamma_deviance"] -
        hybrid_row["mean_gamma_deviance"])
    rmse_gain = float(structural_row["rmse"] - hybrid_row["rmse"])
    corr_gain = float(hybrid_row["corr"] - structural_row["corr"])
    hybrid_material_gain = (
        not math.isnan(deviance_gain)
        and not math.isnan(rmse_gain)
```

```
        and hybrid_row["mean_gamma_deviance"] <
            structural_row["mean_gamma_deviance"]
        and hybrid_row["rmse"] < structural_row["rmse"]
        and hybrid_row["corr"] >= structural_row["corr"]
        and (deviance_gain >= 0.01 or rmse_gain >= 0.5)
    )
    selected_name = "gamma_hybrid_temporal" if hybrid_material_gain else
        "gamma_structural"
    train_all.unpersist()
    test_all.unpersist()
    train_pos.unpersist()
    test_pos.unpersist()
    return comparison, {
        "selected_name": selected_name,
        "selected_spec": spec_map[selected_name],
        "structural_spec": spec_map["gamma_structural"],
        "hybrid_spec": spec_map["gamma_hybrid_temporal"],
        "structural_row": structural_row,
        "hybrid_row": hybrid_row,
        "hybrid_material_gain": hybrid_material_gain,
        "improvement": {
            "mean_gamma_deviance_gain": deviance_gain,
            "rmse_gain": rmse_gain,
            "corr_gain": corr_gain,
        },
    }


def refit_selected_model(feature_df: DataFrame, spec: CandidateSpec):
    train_all = feature_df.filter(F.col("year") == 2022)
    test_all = feature_df.filter(F.col("year") == 2023)
    train_df = train_all.filter(F.col("AQI") > 0) if spec.use_positive_only else
        train_all
    test_df = test_all.filter(F.col("AQI") > 0) if spec.use_positive_only else
        test_all
    model, assembler = fit_glm(train_df, spec)
    pred_df = (
        model.transform(assembler.transform(test_df))
        .withColumn("prediction", F.greatest(F.col("prediction"), F.lit(EPS)))
        .persist(StorageLevel.DISK_ONLY)
    )
    pred_df.count()
    return model, pred_df


def write_outputs(
    comparison: pd.DataFrame,
    selected_bundle: dict[str, Any],
    feature_df: DataFrame,
    metadata: dict[str, Any],
) -> dict[str, str]:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    order = {"null_mean": 0, "gamma_structural": 1, "gamma_hybrid_temporal": 2}
    comparison = (
        comparison.assign(sort_order=comparison["model_name"].map(order).fillna(99))
        .sort_values(["sort_order", "mean_gamma_deviance", "rmse"],
            na_position="last")
        .drop(columns="sort_order")
        .reset_index(drop=True)
    )
    comparison_path = OUTPUT_DIR / "q4_model_comparison.csv"
    comparison.to_csv(comparison_path, index=False)

    structural_spec: CandidateSpec = selected_bundle["structural_spec"]
    structural_model, structural_pred_df = refit_selected_model(feature_df,
        structural_spec)
```

```
    coef_pdf = coefficient_table(structural_model, structural_spec.feature_cols)
    coef_path = OUTPUT_DIR / "q4_coefficients.csv"
    coef_pdf.to_csv(coef_path, index=False)

    pred_pdf = (
        structural_pred_df.select(
            "AQI",
            "prediction",
            "month",
            "region",
            "division",
            "county_mean_aqi_2022",
            "county_summer_mean_2022",
            "Defining_Parameter",
        )
        .toPandas()
    )

    plot_eda(feature_df)
    plot_state_map(feature_df)
    plot_diagnostics(pred_pdf)
    effect_summary = plot_effects(pred_pdf, coef_pdf, structural_spec)

    data_summary = collect_data_summary(feature_df, metadata)
    predictive_row = comparison.loc[comparison["model_name"] ==
        selected_bundle["selected_name"]].iloc[0].to_dict()
    null_row = comparison.loc[comparison["model_name"] ==
        "null_mean"].iloc[0].to_dict()

    summary = {
        "main_scientific_model": structural_spec.name,
        "predictive_model": selected_bundle["selected_name"],
        "chosen_model": selected_bundle["selected_name"],
        "final_specification": {
            "name": selected_bundle["selected_spec"].name,
            "family": selected_bundle["selected_spec"].family,
            "block": selected_bundle["selected_spec"].block,
            "description": selected_bundle["selected_spec"].description,
            "feature_cols": selected_bundle["selected_spec"].feature_cols,
            "interpretation": selected_bundle["selected_spec"].interpretation,
            "eligible_for_final":
                selected_bundle["selected_spec"].eligible_for_final,
        },
        "baseline_model": null_row,
        "chosen_metrics": predictive_row,
        "structural_model": {
            "specification": {
                "name": structural_spec.name,
                "family": structural_spec.family,
                "block": structural_spec.block,
                "description": structural_spec.description,
                "feature_cols": structural_spec.feature_cols,
                "interpretation": structural_spec.interpretation,
            },
            "metrics": selected_bundle["structural_row"],
        },
        "hybrid_model": {
            "specification": {
                "name": selected_bundle["hybrid_spec"].name,
                "family": selected_bundle["hybrid_spec"].family,
                "block": selected_bundle["hybrid_spec"].block,
                "description": selected_bundle["hybrid_spec"].description,
                "feature_cols": selected_bundle["hybrid_spec"].feature_cols,
                "interpretation": selected_bundle["hybrid_spec"].interpretation,
            },
            "metrics": selected_bundle["hybrid_row"],
```

```
            "material_gain": selected_bundle["hybrid_material_gain"],
        },
        "predictive_comparison": {
            "selected_name": selected_bundle["selected_name"],
            "hybrid_material_gain": selected_bundle["hybrid_material_gain"],
            "improvement": selected_bundle["improvement"],
        },
        "data_summary": data_summary,
        "effect_summary": effect_summary,
        "model_comparison": comparison.to_dict(orient="records"),
    }
    summary_path = OUTPUT_DIR / "q4_summary.json"
    summary_path.write_text(json.dumps(summary, indent=2, default=float),
        encoding="utf-8")
    structural_pred_df.unpersist()

    return {
        "summary_path": str(summary_path),
        "comparison_path": str(comparison_path),
        "coefficients_path": str(coef_path),
        "output_dir": str(OUTPUT_DIR),
    }


def main() -> dict[str, str]:
    spark = spark_session()
    try:
        raw_df = load_data(spark)
        feature_df, metadata = prepare_feature_bank(raw_df)
        comparison, selected_bundle = fit_all_models(feature_df, metadata)
        paths = write_outputs(comparison, selected_bundle, feature_df, metadata)
        print(f"Main scientific model: {selected_bundle['structural_spec'].name}")
        print(f"Predictive model: {selected_bundle['selected_name']}")
        print(f"Outputs saved to {paths['output_dir']}")
        return paths
    finally:
        spark.stop()


if __name__ == "__main__":
    main()
```

Listing 12: Full source code for Question 4 (`src/q4_pyspark_public_dataset.py`)